
计算机系统前沿 课程概述



崔立骁

cuilx@nbjl.nankai.edu.cn

Slides部分来源于Animesh Trivedi,
Storage Systems (StoSys)
XM_0092, 以及清华大学存储系统课程



Parallel and Distributed
Software Technology Lab



Nankai - Baidu
Joint Laboratory



目录

- 课程介绍
- 影响计算机系统设计的因素
 - 硬件变革的影响
 - 软件变革的影响
 - 存储和计算的关系
- 后续课程预告



课程团队

- 授课教师

- 李雨森

- 计算系统，深度学习系统

- 崔立骁

- 存储系统

- 课程飞书群



南开计算机系统前沿 2026...
南开大学



仅限企业内部成员加入

该二维码 1 年内 (2027 年 2 月 24 日前) 有效

课程要求



- 先导课
 - 高级语言程序设计
 - 操作系统
 - 计算机体系结构
- 成绩
 - 45%: 平时作业（不能抄袭）、出勤等
 - 15%: 前沿论文阅读和课堂展示（学期末）
 - 40%: 期末大作业

课程要求



- 平时作业
 - 2-4次，一些简单的计算机系统实践和问题解答
- 课堂展示和讨论
 - 每一个课程主题之后给出论文阅读列表（计算机系统前沿文章）
 - 选取1篇文章进行深入阅读和思考
 - 学期末随堂展示（5分钟）& 提交阅读报告

课程要求



- 大作业
 - 多个选题，任选其一
 - (1) 向量存储系统的实现和I/O优化
 - (2) 大模型推理系统的KV cache卸载以及优化
 - (3) agent系统优化（待定）
 -
- 个人独立完成，提交报告和代码

课程要求



- 人工智能使用要求

- 在每次作业的报告最后，附加**人工智能使用情况说明**
- 可以使用AI工具，但需要在情况说明中提交 **与AI工具的交互记录**

- 作业格式要求

- 简化表达是科学研究能力的体现
- 大作业要求报告页数**不能超过20页**（不包含人工智能使用情况说明）
- 前沿论文阅读报告**不能超过5页**（不包含人工智能使用情况说明）
- 一些作业不设置标准答案，你只需要根据实际软硬件情况探索系统优化空间

AI时代中还没有被打败的领域

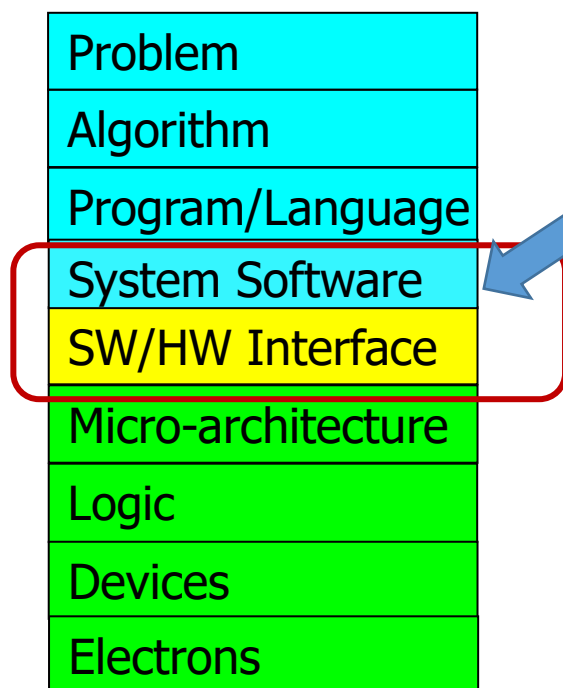


- code is cheap, show me the talk
- AI已经可以达到IMO金牌水平，CS学习是否还有意义？
- 计算机系统的复杂性仍是AI难以完成的（至少目前是）
- 系统——“抽象的艺术” 对底层抽象，对上层应用隔离
- AI运行的基座
- 从计算机诞生开始存在，随时代变革而不消失
 - Unix, Google大数据三件套, MLsys

课程目标



计算机层次结构



本课程主要关注的领域

操作系统和计算机体系结构课程中未涉及的知识

有志于未来从事此方向的学术和工业界工作

课程目标



- **目标1：理解计算机系统前沿领域的核心挑战与演进方向**，了解前沿计算机系统的主流优化方案。并且能够利用所学知识对复杂的计算机系统解决方案进行比较与综合，从而解决复杂的计算机工程问题。
- **目标2：通过所学知识和相关文献阅读，研究前沿计算机系统中不同设计所考虑的权衡因素，理解硬件与应用变革对计算机系统解决方案的影响，具备分析复杂计算机工程问题并得到结论的能力。**

课程目标



- 目标3： **掌握对计算机系统进行评估与解决方案优化的能力。**能够分析存储、缓存、深度学习训练推理等系统的性能瓶颈。掌握从硬件特性到软件栈各层的协同设计原则，并能够针对不同场景和特定需求提出优化的系统解决方案，在设计环节中体现创新意识，考虑社会、健康、安全、法律、文化以及环境等因素。
- 目标4：完成课堂讨论，完成计算机前沿系统调研和演讲报告等工作。

相关资料



- 课程教材和参考书
- 《数据密集型应用系统设计》（存储）
- Martin Kleppmann著，赵军平等译
- 《机器学习系统：设计和实现》
- 麦络，董豪 著

前沿方向演进速度快，教科书不是必须

相关资料



- 会议:
- 系统方面: OSDI、SOSP、FAST、USENIX ATC、EuroSys、MLsys、NSDI等会议的论文
- 体系结构方面: ASPLOS、HPCA、ISCA、MICRO等会议的最新文章
- 数据库方面: SIGMOD、VLDB等会议上**计算机系统**相关的文章
- 期刊:
- ACM Transactions on Storage (TOS)的文章
- IEEE Transactions on Computer (TC)的相关文章
- IEEE Transactions on Parallel and Distributed Systems (TPDS)的相关文章

关注前沿方向的研究成果对本课程更有意义

相关资料



- 相关研究的团队
- UC Berkeley Sky Computing Lab @ UC Berkeley EECS
- <https://sky.cs.berkeley.edu/>
- Parallel Data Lab @ Carnegie Mellon University
- <http://www.pdl.cmu.edu/>
- ADvanced System Lab @ University of Wisconsin—Madison
- <http://www.cs.wisc.edu/adsl/>
- MIT CSAIL Parallel and Distributed Operating Systems Group
- <https://pdos.csail.mit.edu/>

关注前沿方向的研究成果对本课程更有意义

AI时代暂时还没有被打败的领域



- code is cheap, show me the talk
- AI已经可以达到IMO金牌水平，CS学习是否还有意义？
- 计算机系统的复杂性仍是AI难以完成的（至少目前是）
- 系统——“抽象的艺术” 对底层抽象，对上层应用隔离
- AI运行的基座
- 从计算机诞生开始存在，随时代变革而不消失
 - Unix, Google大数据三件套, MLsys, AI主导进化的系统（这一天本课程将无意义）
- AI的知识面极度广阔，追求分数或许已经失去意义
- 本次课程——**人类**对CS的设计思想，掌握AI编写的代码





目录

- 课程介绍
- **影响计算机系统设计的因素**
 - 硬件变革的影响
 - 软件变革的影响
 - 存储和计算的关系
- 后续课程预告



影响计算机系统设计因素



机器学习

大数据应用

web应用

流媒体应用

应用

存储

网络

计算

计算机系统

硬件



GPU



HDD



ASIC



SSD



Persistent Memory

硬件变革的影响

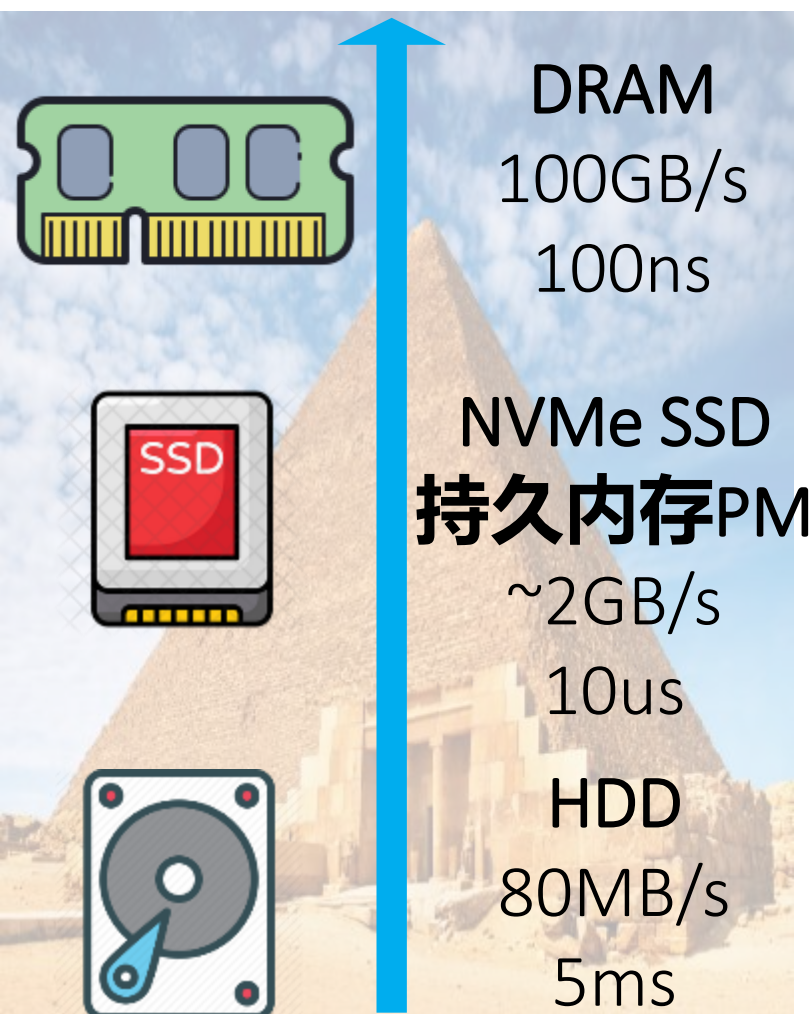


- 存储硬件革新对系统设计的影响
 - 更高速度的SSD设备
 - 改变存储体系结构层次的持久内存
 - 更大容量、速度更慢的叠瓦式硬盘
- 计算硬件的革新
 - CPU到GPU，众核计算设备的兴起
 - 专用领域计算加速器



存储硬件变革的发展趋势

- 存储金字塔层级已不再是个金字塔



时延接近

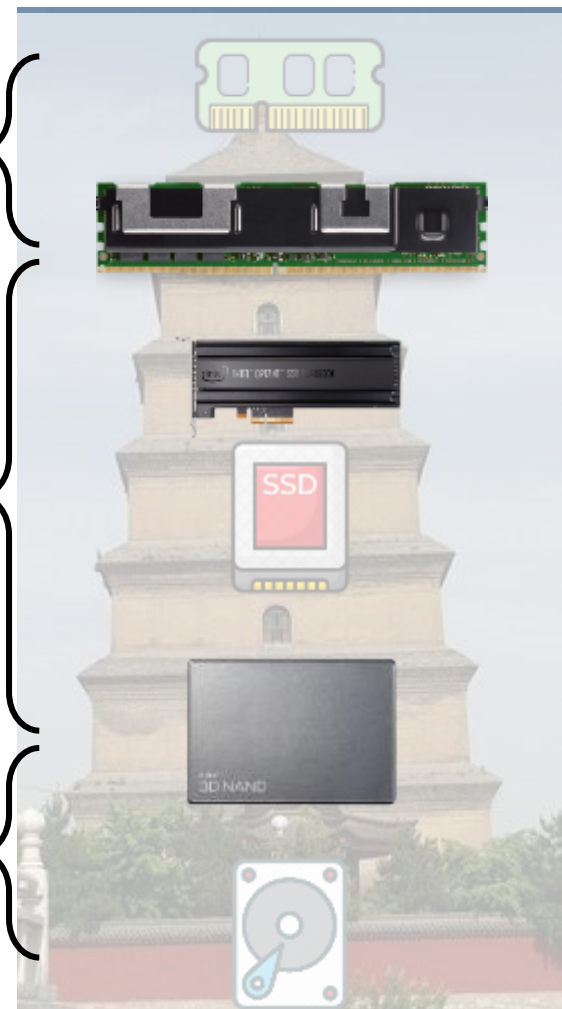
~100ns

写带宽接近

单设备~2GB/s

容量接近

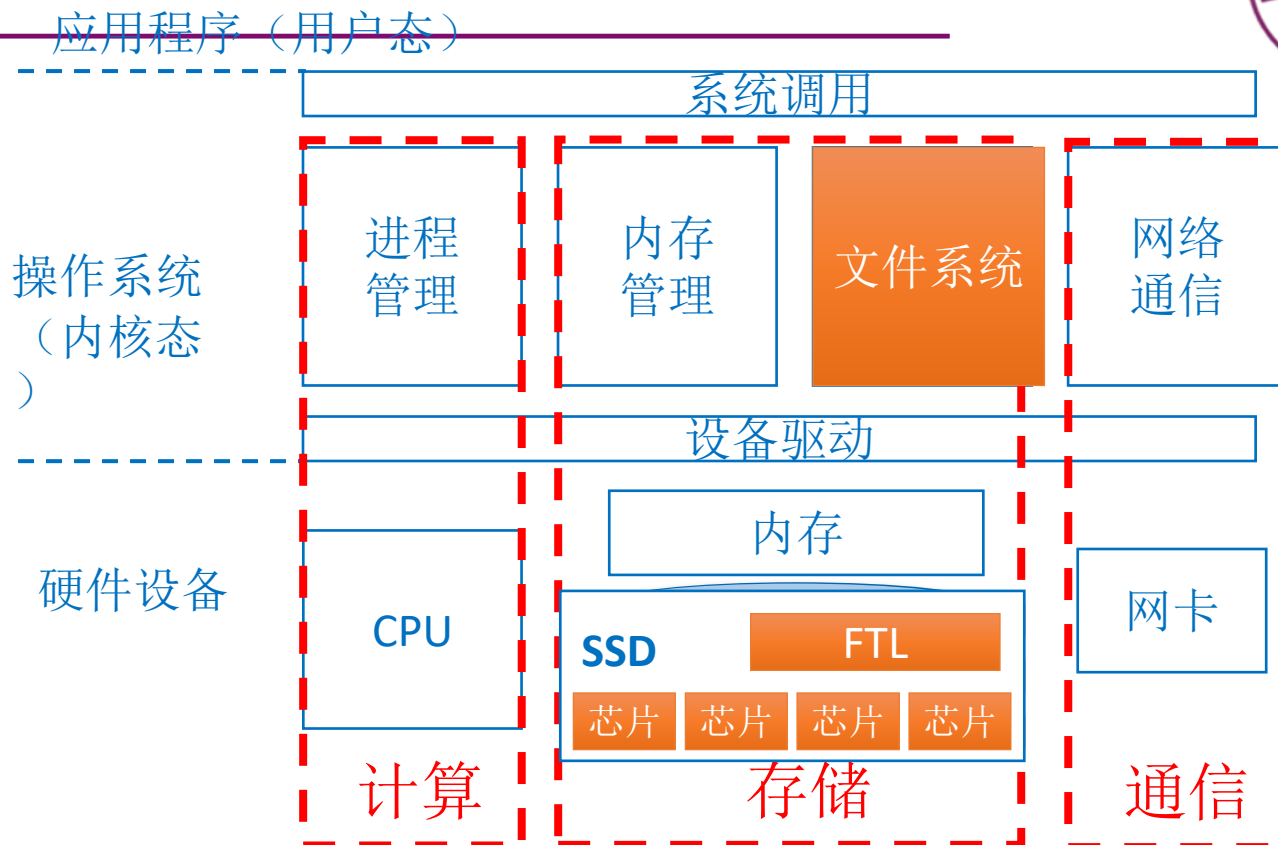
单设备~1TB



数量级差异!

层级界限模糊

SSD闪存与文件系统



- 在计算机系统的三大基石（计算、存储、通信）中，**唯有存储仍包含机械式部件操作**
- 电子式存储器件 - **闪存** - 是电子式计算机发展的趋势
- 然而，**系统软件（文件系统）积重难返，未能充分发挥闪存优势**

SSD闪存与文件系统

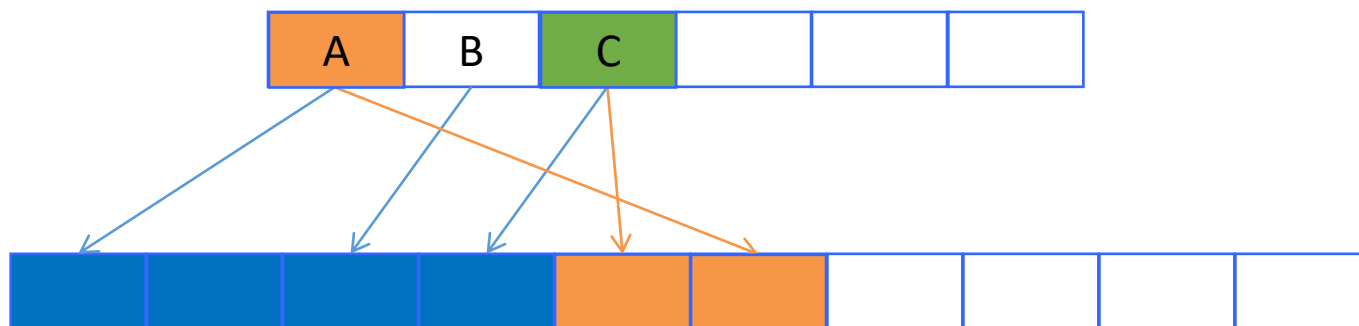


- 闪存

- 电子式存储器件：无机械式部件

- 读写特性

- 异地更新：不可覆盖写
- 页粒度读写，块粒度擦除：如4KB页面，256KB闪存块



- 耐久性

- SLC (100,000) -> MLC (10,000) -> **TLC (1,000)**

SSD闪存与文件系统



- 1984年，日本东芝公司
- 嵌入式存储
- SD卡
- 固态硬盘 SSD
- PCIe 闪存加速卡
- 全闪存阵列
-



SSD 不再适用传统的HDD优化



- ~~更大的读写单元~~：减小随机性，充分利用顺序读写带宽 (可以用更小读写单元)
- **内存作磁盘cache**：利用数据访问的空间/时间局部性，用更快的内存加快IO。(作用减弱)
- **透明压缩**：压缩磁盘块可以减少IO带宽，提高性能。(无法提升IO性能，只能节省存储空间)
- ~~IO合并~~：将多次空间连续但是提交时间乱序的小IO合为一次连续大IO。(反而增加延迟，越高性能SSD越不需要合并)
- **使用Log structured 方式优化写性能**：用追加的方式将随机写转为顺序写。(意外收获：用在SSD上可以减少写放大)

SSD闪存与文件系统



闪存介质的访问，呈现低延迟、读写不对称的特点，随机读写性能较硬盘提升很高。传统针对硬盘优化设计的软件系统直接用于闪存时，一方面带来了不必要的冗余功能，另一方面隐藏了闪存可能带来的优势。

● 传统文件系统存在的问题

1. 冗余工作：文件系统中从文件逻辑块到设备物理块的映射，与FTL中逻辑地址到物理地址的映射 → 双层映射
2. 语义缺失：设备不能理解数据页面间的关系，难以优化数据分布和感知文件系统的操作 → 数据删除
3. 特性缺失：闪存设备的异地更新与文件系统的原子性操作 → 两

次写

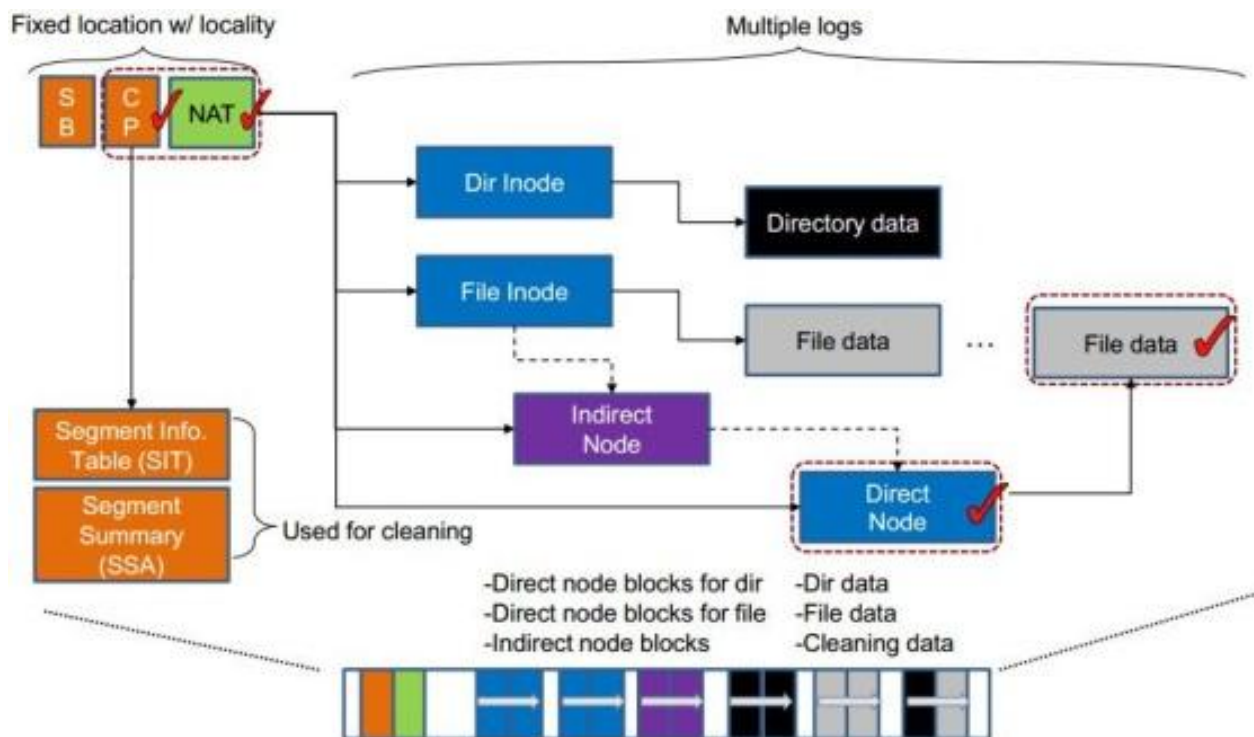


SSD闪存与文件系统



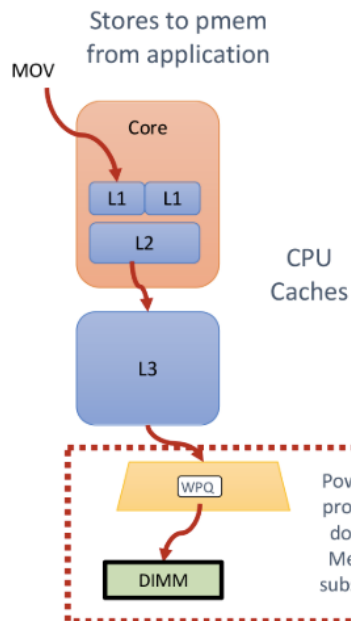
■ F2FS——针对SSD优化的日志型文件系统（FAST'2015）

- ✓ 冷热数据分组提高GC效率。
- ✓ 地址对齐，FS的GC单位与SSD保持一致。

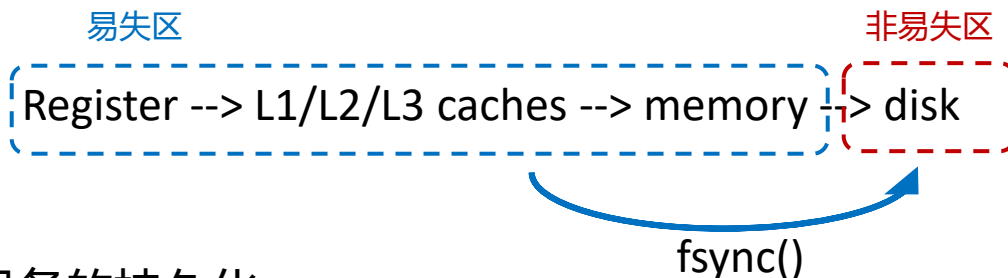




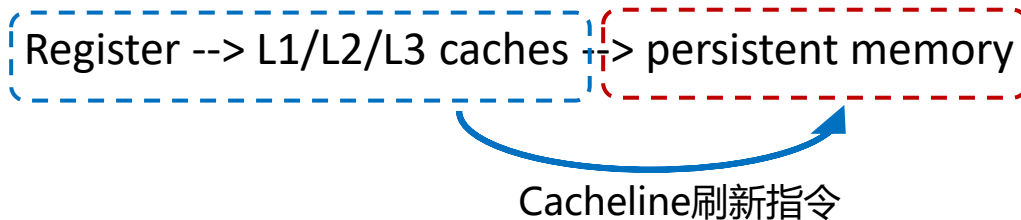
持久内存的持久化挑战



- 传统存储设备的持久化:



- PM设备的持久化:



- PM的崩溃一致性

- x86平台的Failure atomicity 仅为8字节。
- 注意: 虽然cache以64字节的cacheline大小进行cache到内存的数据传输, 但硬件上仍然分8次(每次8字节)进行传输, 所以即使是同一cacheline, 也会被系统崩溃或断电打断。
- 任何大于8字节的持久化操作, 都要用额外的崩溃一致性机制保证整个存储系统的一致性。常用的方法有写时复制、日志等方法。



持久内存与键值对系统

挑战：如何设计全新的系统架构，最大程度发挥硬件性能
索引优化

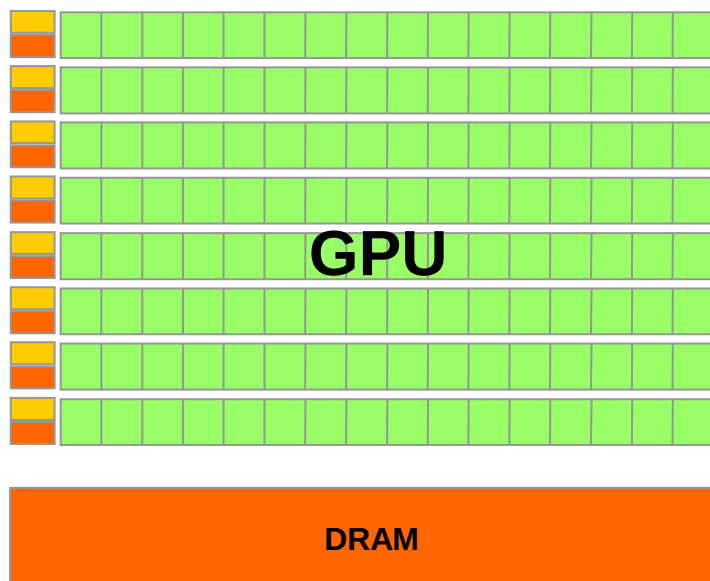
- 索引是KV系统的最重要的组件（B+树，hash表，基数树，跳表等）
- PM给构建持久性，高性能索引提供机会
- 主要关注问题： ①索引更新时的崩溃一致性；
②减少索引结构变化对PM的写开销

	Hash表	B+树	基数树
代表工作	levelhash@OSDI'18, CCEH@FAST'19, Clevel@ATC'20, Dash@VLDB'20	NV-tree@FAST'15, FPtree@SIGMOD'16 Wb+tree@VLDB'15 FAST&FAIR@FAST'18	WORT@FAST'17 HART@IPDPS'19 ROART@FAST'21
崩溃一致性解决方案	8字节原子写	8字节原子写, Log, copy-on-write	8字节原子写, copy-on- write
PM写开销解决方案	重构hash表结构, 减少 Rehash开销	叶结点内部乱序存储, 内部节点存放在DRAM, 根据体系结构特性减少 clflush数量	重新设计结构减少 clflush数量

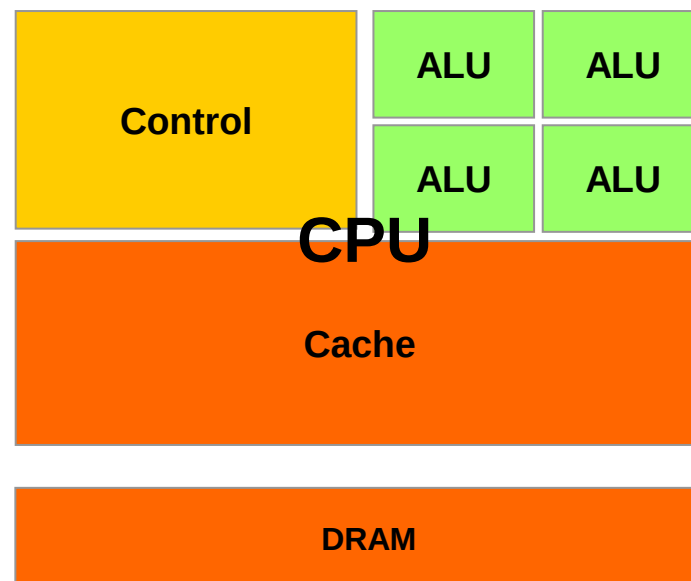
GPU带来的系统变化



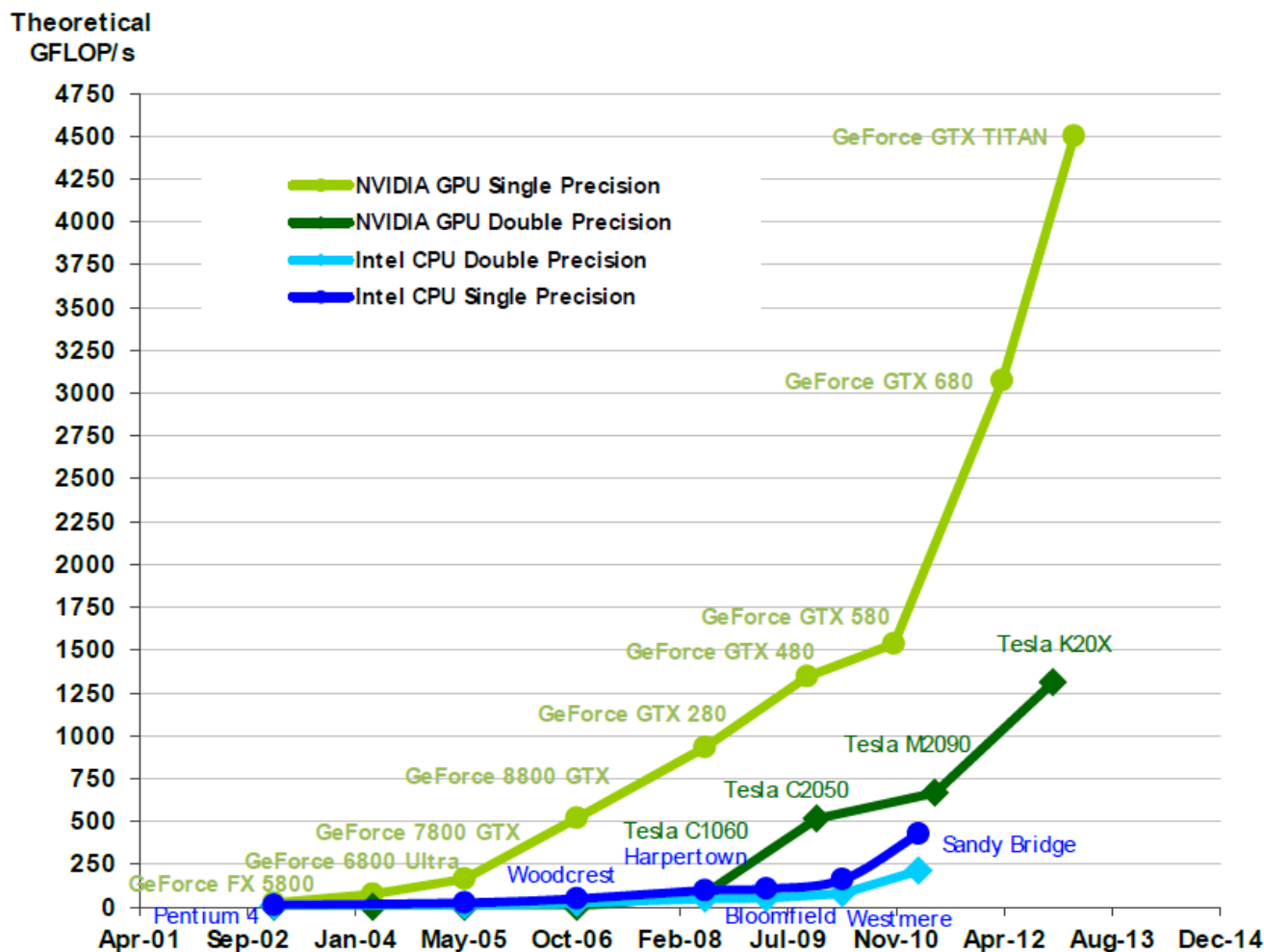
GPU设计
面向吞吐率



CPU核心设计
面向延迟



GPU带来的系统变化



CUDA C Programming Guide, Nvidia, 2013.



Parallel and Distributed
Software Technology Lab

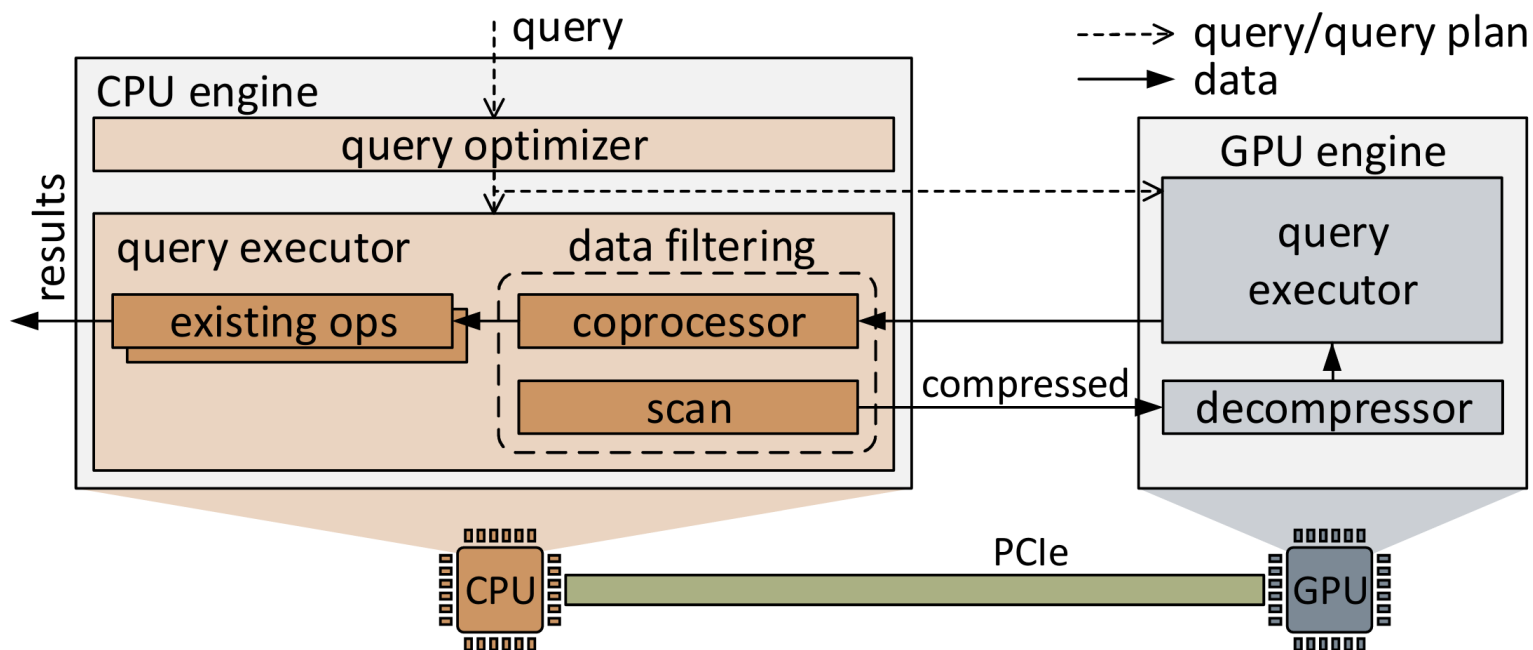


Nankai-Baidu
Joint Laboratory

GPU带来的系统变化



GPU 数据库

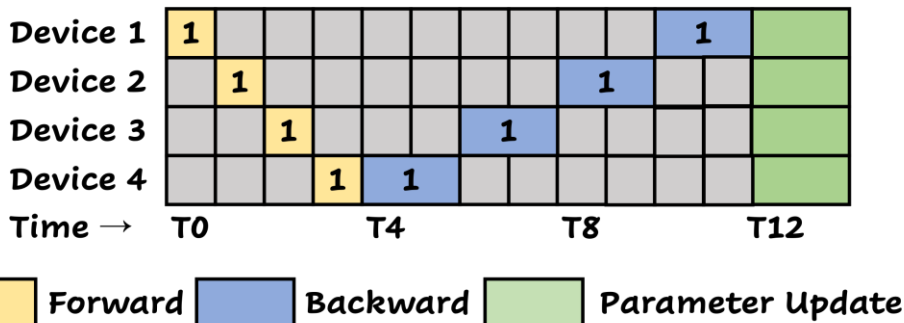
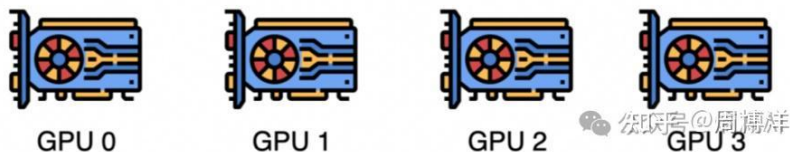
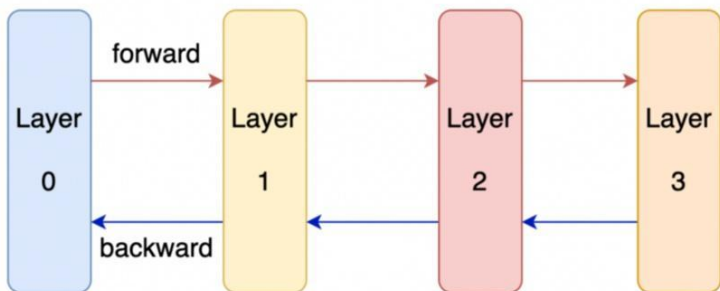


Scaling GPU-Accelerated Databases beyond GPU Memory Size. VLDB2025.

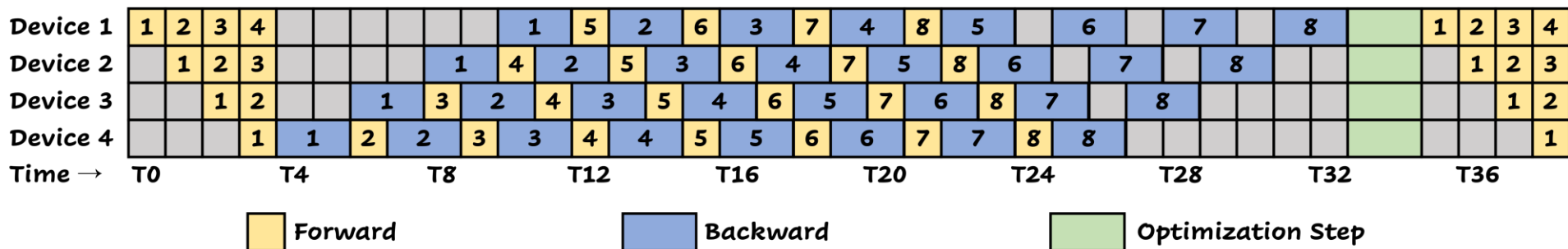
GPU带来的系统变化



大模型分布式训练



未经优化的流水线



1F1B的流水线优化

应用变革的影响



- 大数据应用对系统设计的影响
 - Google大数据三驾马车
- Web应用催生的系统革新
 - 内存键值对缓存系统
- 大模型应用催生的机器学习
 - 大模型推理引擎和并行训练系统

大数据应用与大数据系统



数据量爆炸 (Volume)

数据规模从TB级跃升至PB、EB级。



数据多样化 (Variety)

数据来源多、类型多、结构复杂



数据处理实时性 (Velocity)

数据生成速度快，用户期望快速响应。



传统单点集中式系统难以处理

大数据应用与大数据系统



Google大数据时代 “三驾马车”

GFS 分布式文件系统

MapReduce 并行计算

Bigtable 结构化存储

The Google File System [SOSP'03]

MapReduce: Simplified Data Processing on Large Clusters [OSDI'04]

Bigtable: A Distributed Storage System for Structured Data[OSDI'06]

大数据应用与大数据系统



GFS: 利用大量廉价的商用服务器构建可扩展、高可用的分布式存储集群。

Master Node (主节点)

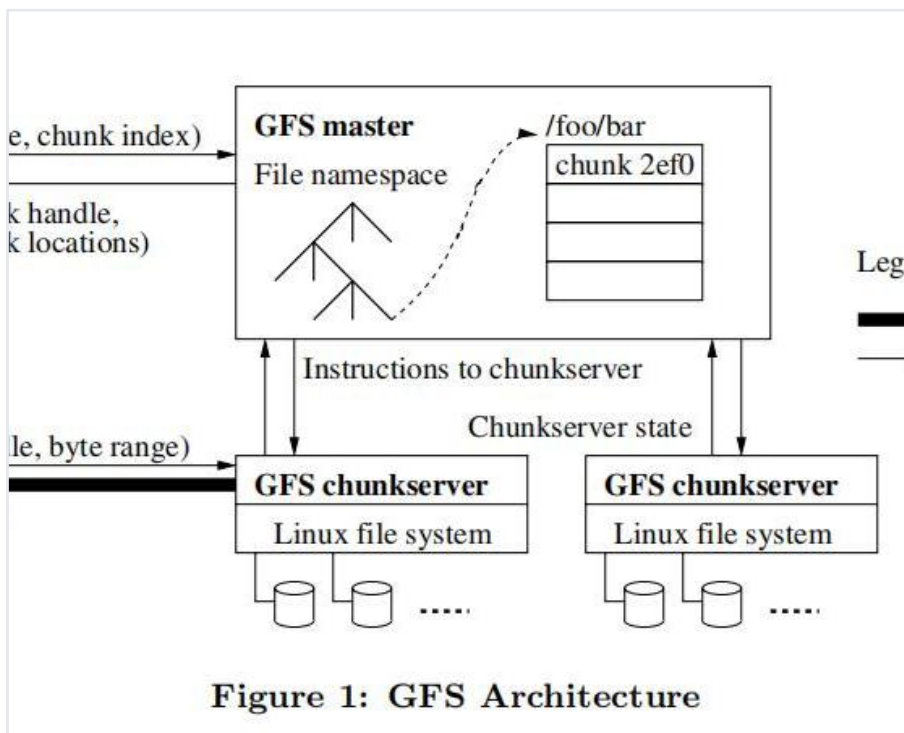
- 管理元数据（文件名、目录结构、数据块映射）

ChunkServer (数据块服务器)

- 存储实际数据块（默认64MB），响应读写请求

Client (客户端)

- 与Master交互获取元数据，定位数据块位置



大数据应用与大数据系统



MapReduce: 分而治之

- **任务分解:** 将大规模计算任务自动分解为多个相互独立的Map子任务, 在集群节点上并行执行。
- **结果合并:** 收集各节点的计算结果, 通过Reduce任务进行聚合处理, 最终得到全局结果。

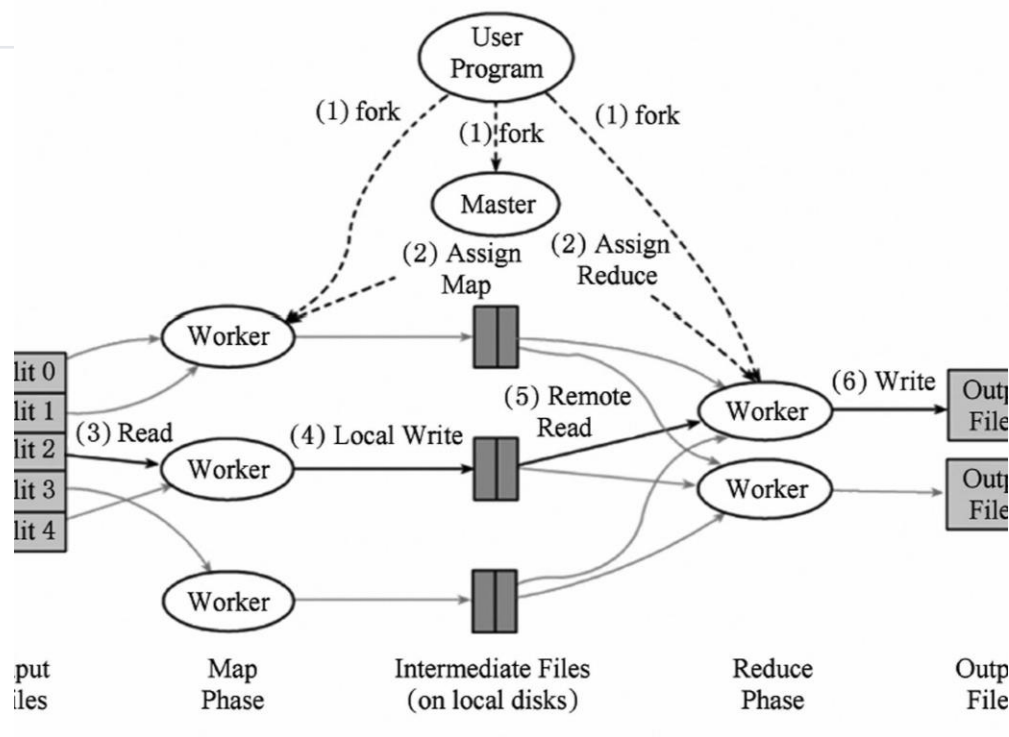


Fig. 2 Execution overview of MapReduce.

图 2 MapReduce 执行流程图

大数据应用与大数据系统



BigTable：分布式多维排序映射表

构建一个分布式的、稀疏的、多维排序映射表，用于高效管理大规模的结构化数据，支持快速查询与动态扩展。

数据模型映射关系：(Row Key, Column Family: Column Qualifier, Timestamp) -> Value



行 (Row)

数据按 Row Key 全局排序，是数据访问的主索引，决定了数据的物理分布。



列族 (Column Family)

列的分组单位，是访问控制和物理存储的基础，需要预先定义，不能随意修改。



列 (Column)

由“列族:列限定符”组成，属于特定列族，可以动态扩展，适合存储稀疏数据。



值 (Value)

单元格中存储的具体数据。

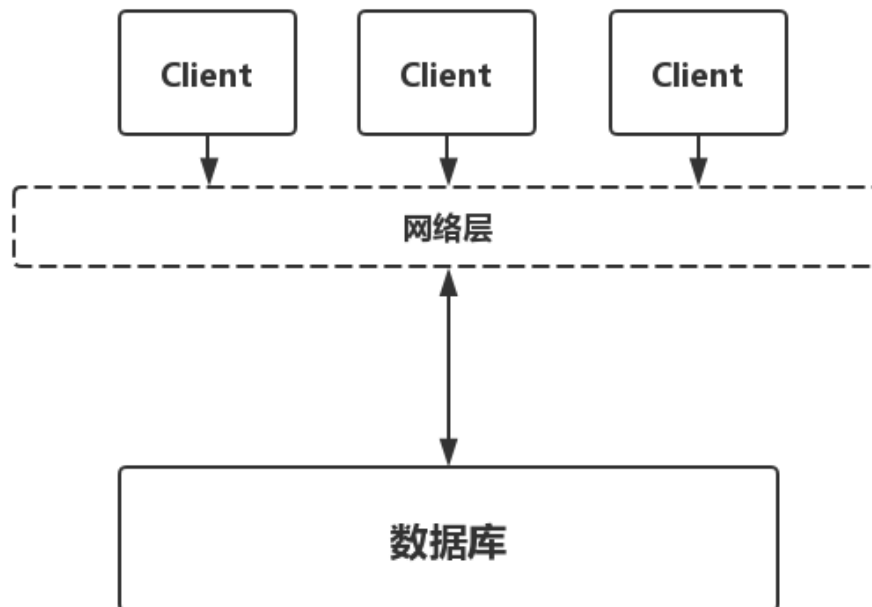
Web应用与内存键值缓存系统



2000年左右web
应用兴起

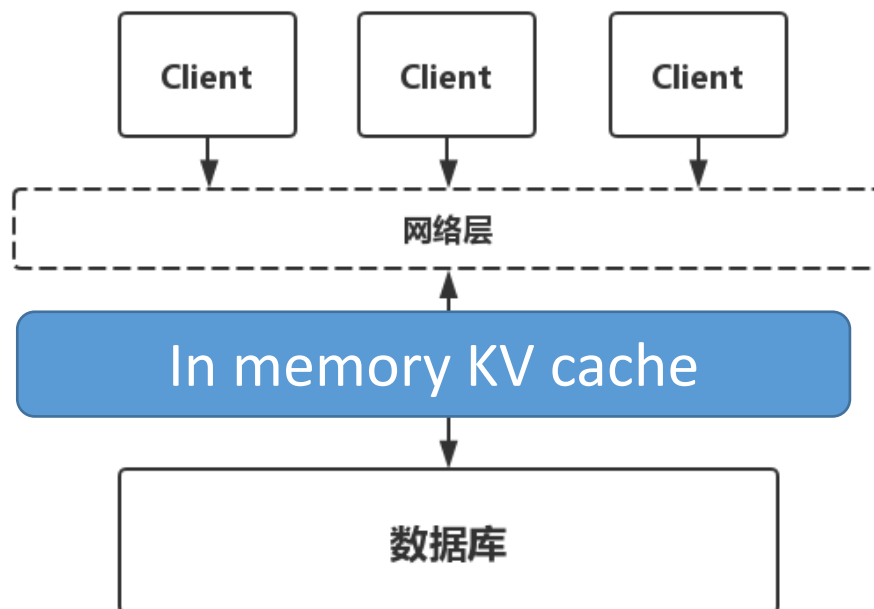
图片来源: 视觉中国 www.vcg.com

Web应用与内存键值缓存系统



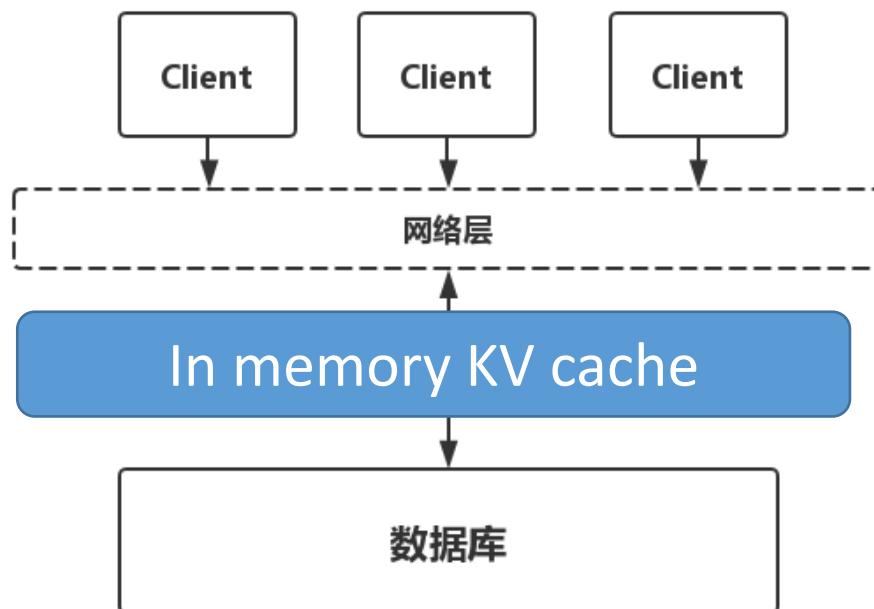
- LiveJournal 的创始人 Brad Fitzpatrick 发现，他的 MySQL 数据库被用户的“页面刷新”请求压垮。
- 90% 的数据库请求都是在读取相同的数据（例如：这篇博客的内容是什么？）

Web应用与内存键值缓存系统



- Memcached-第一个内存键值缓存系统
- 利用多台服务器空闲的内存，临时存储查询结果
- 它不存数据，只缓存数据。让数据库压力骤减的情况下抗住了指数级的用户增长。

Web应用与内存键值缓存系统



- Memcached-第一个内存键值缓存系统
- 利用多台服务器空闲的内存，临时存储查询结果
- 它不存数据，只缓存数据。让数据库压力骤减的情况下抗住了指数级的用户增长。

Web应用与内存键值缓存系统



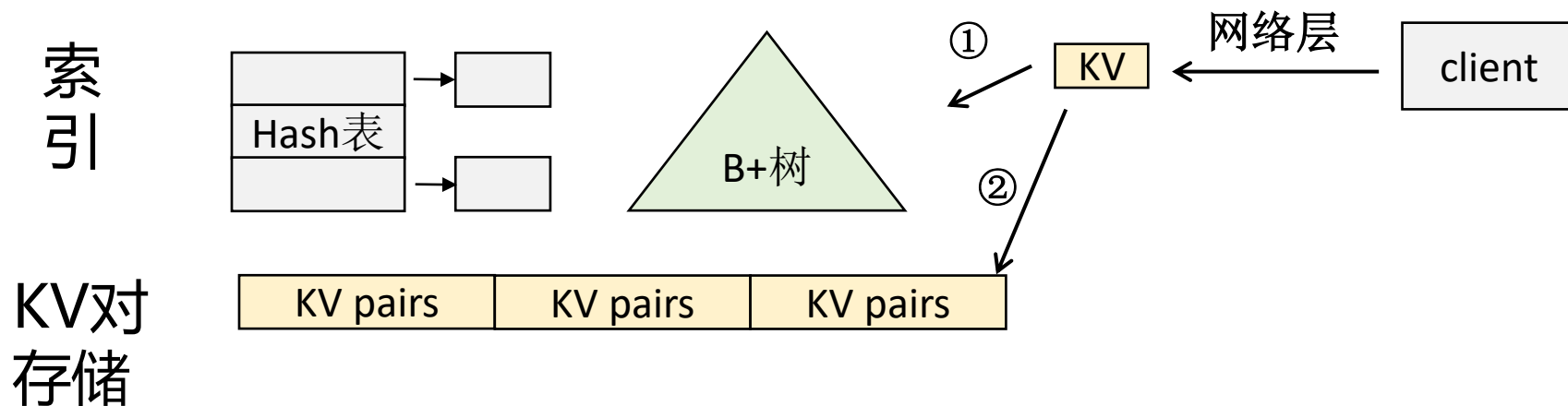
Memory is the new disk.
Disk is the new tape.
——Jim Gray

- 工业界产品：Memcached, Redis, etc.



内存键值对系统工业界应用广泛。几乎所有的互联网公司都有内存键值对团队。例如Meta维护了容量数百TB的的Memcached集群

Web应用与内存键值缓存系统



- 核心是数据索引和KV对存储
- 在本机索引+内存存储基础上增加网络层，用于远程交互
- 数据索引：B+树，hash表，跳表，基数树等
- 内存存储方式：slab分配器，日志结构等

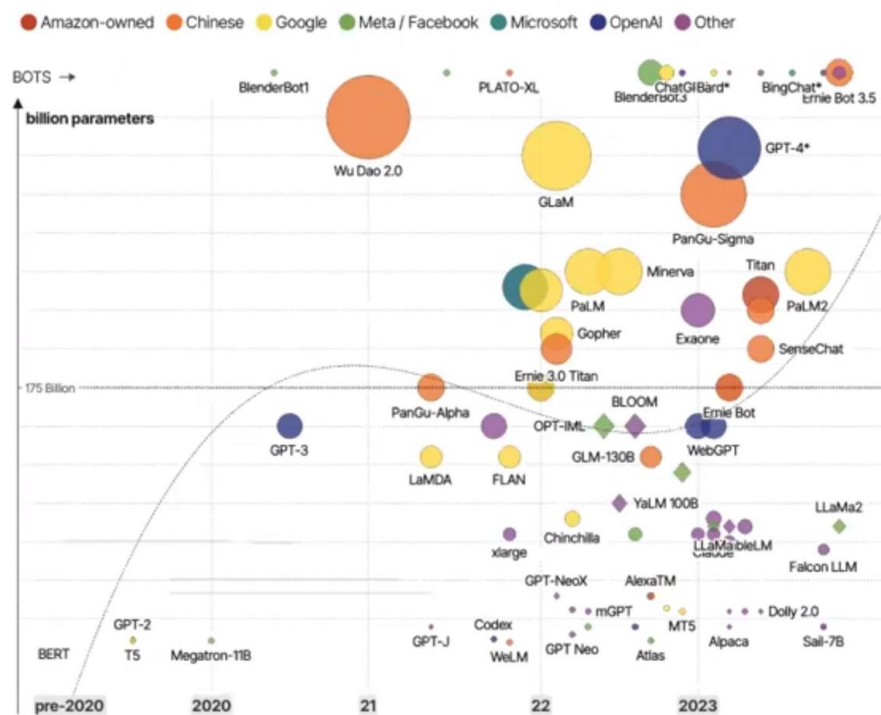
大模型与机器学习系统



AI大模型时代



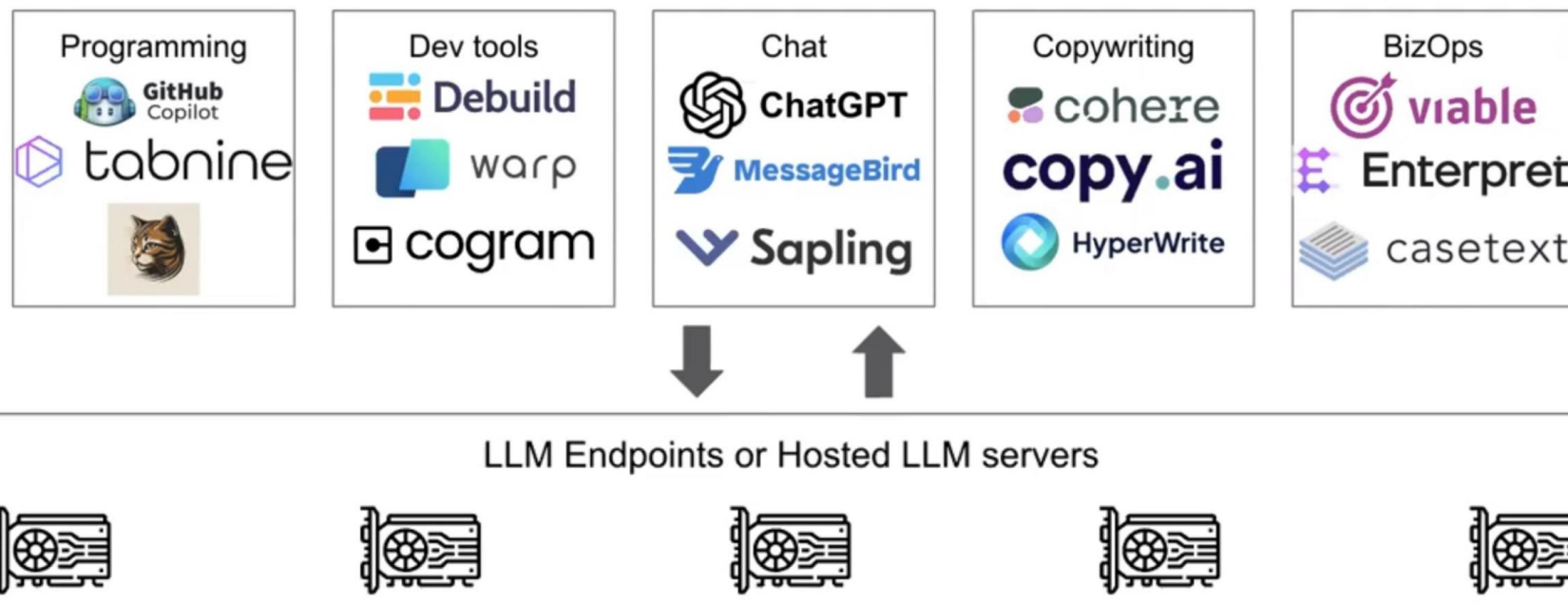
The Rise and Rise of A.I. Large Language Models (LLMs) & their associated bots like ChatGPT



大模型与机器学习系统



大模型应用需要高效的推理系统



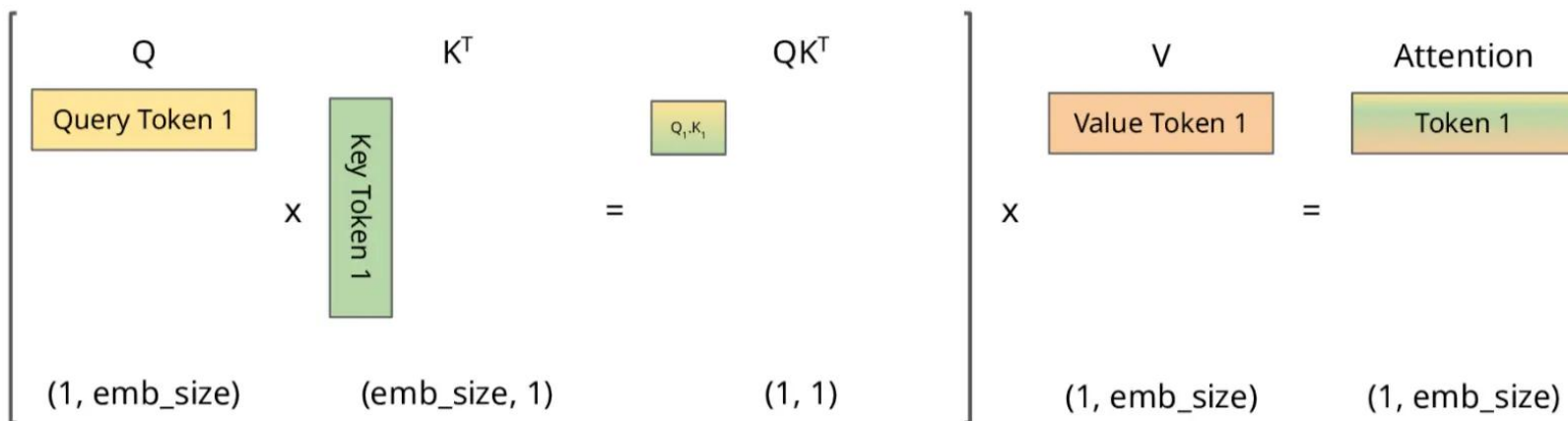
大模型与机器学习系统



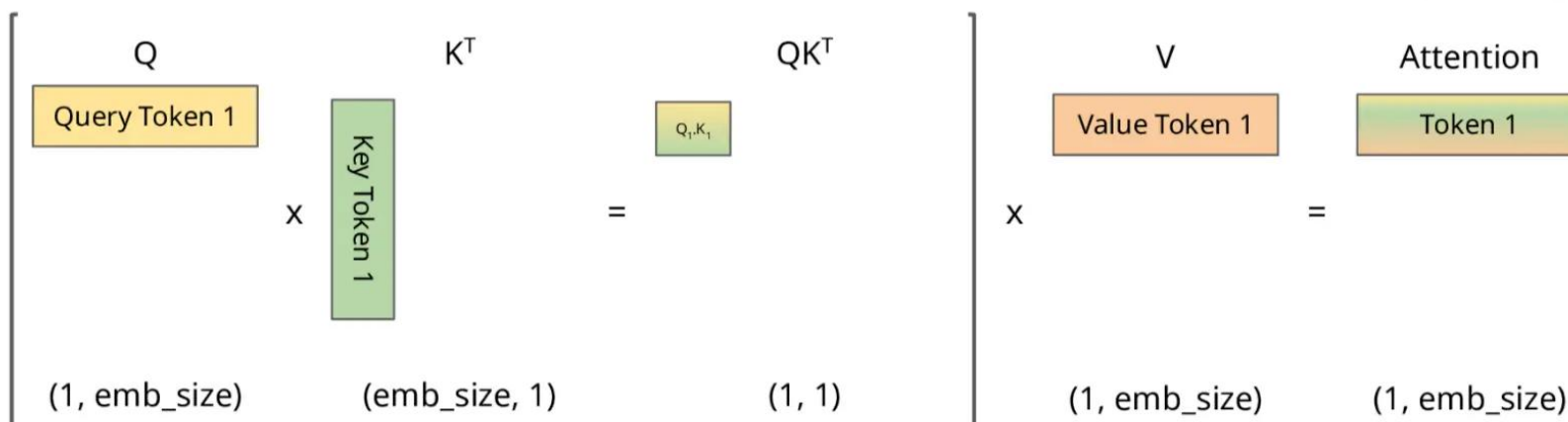
KV cache 避免重复计算

Step 1

Without
cache



With
cache



Values that will be masked Values that will be taken from cache

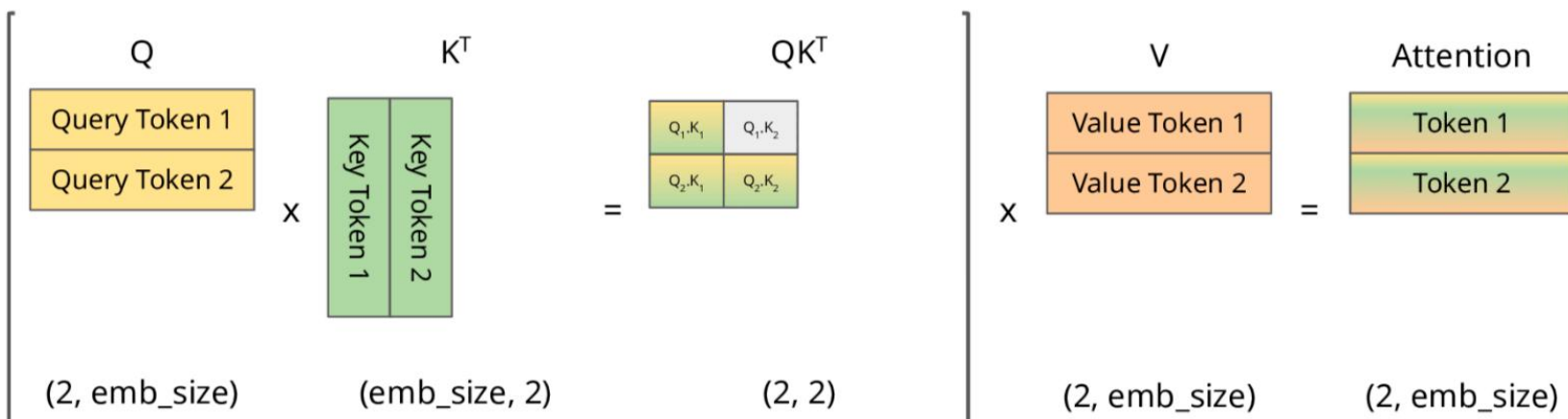
大模型与机器学习系统



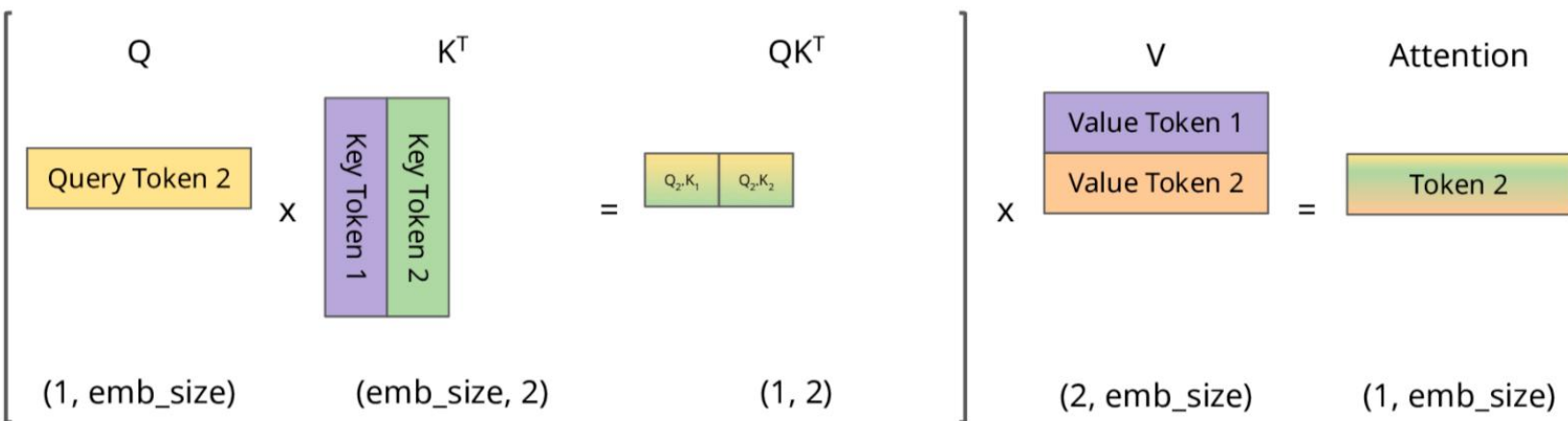
KV cache 避免重复计算

Step 2

Without
cache



With
cache



Values that will be masked Values that will be taken from cache

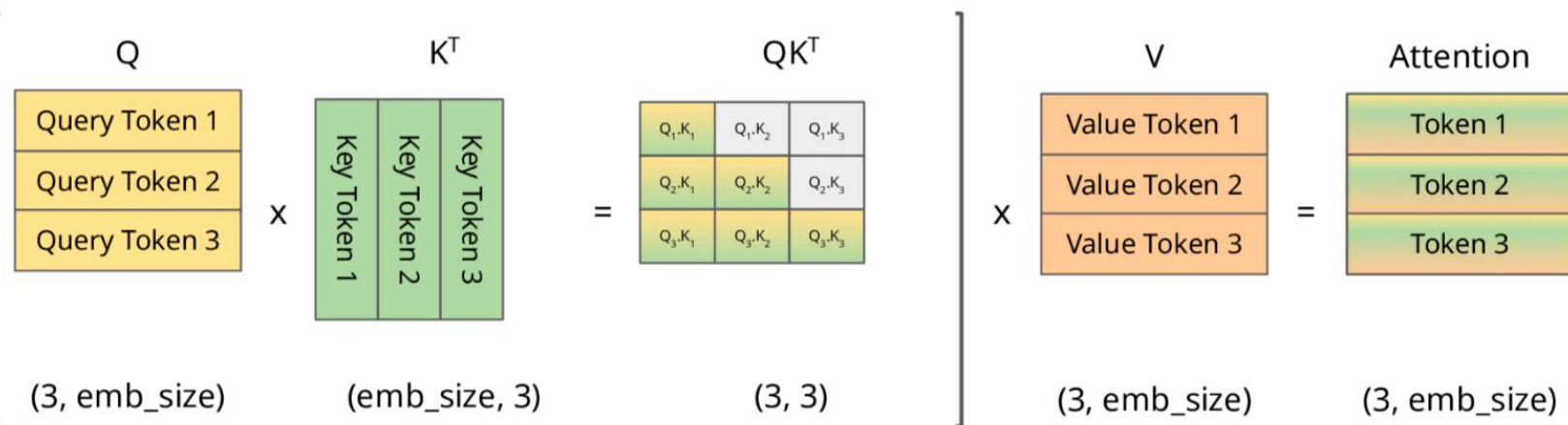
大模型与机器学习系统



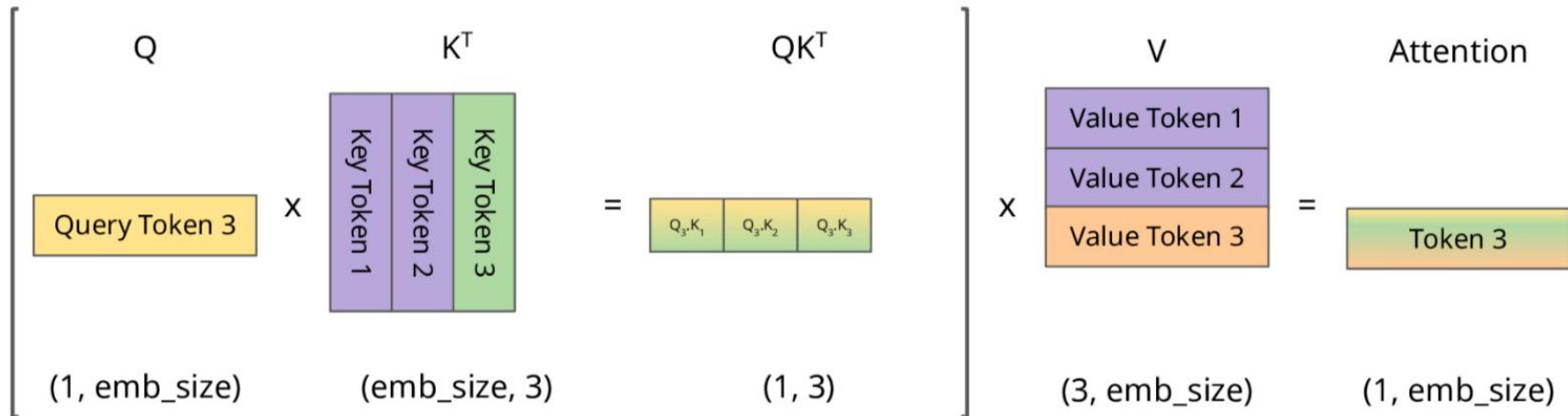
KV cache 避免重复计算

Step 3

Without
cache



With
cache

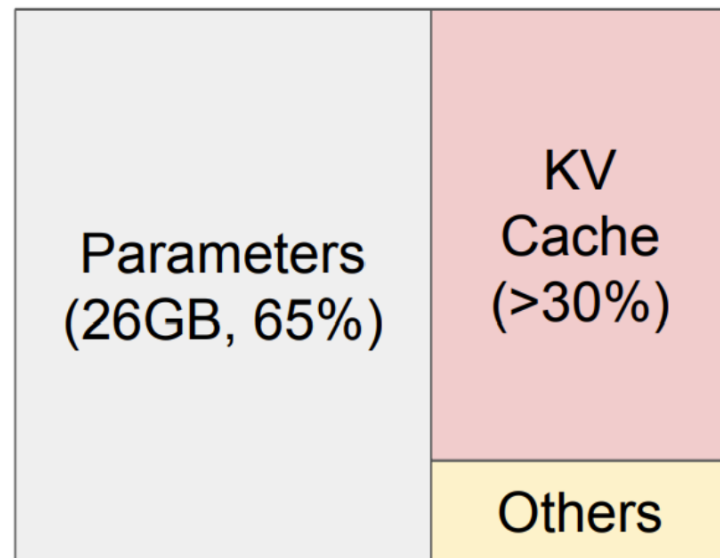
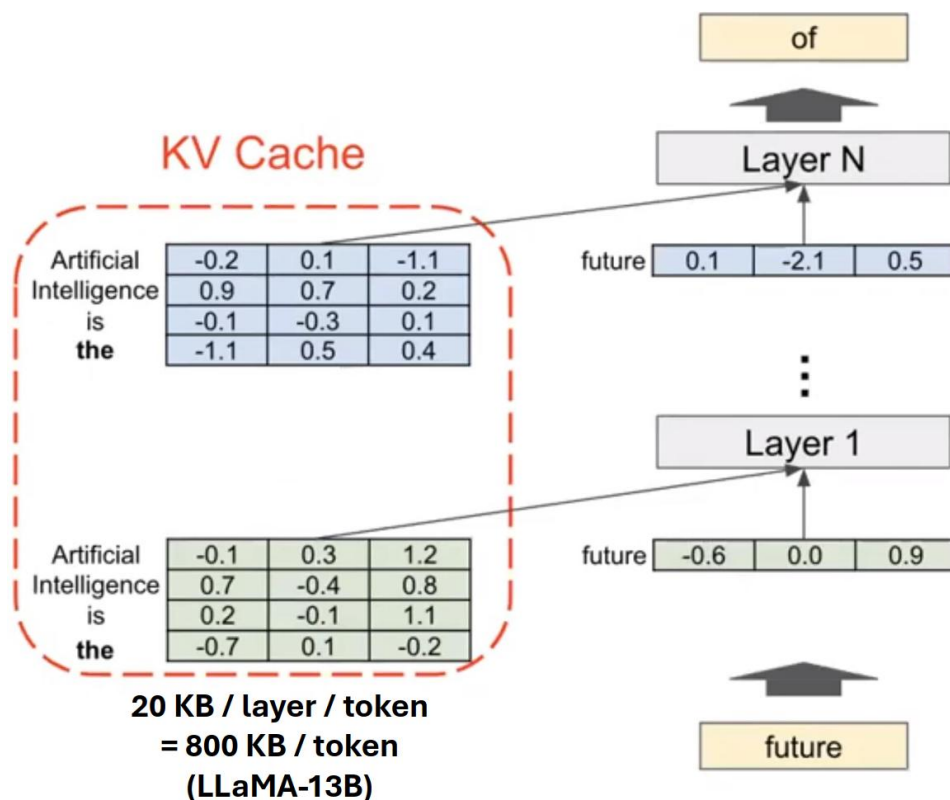


Values that will be masked Values that will be taken from cache

大模型与机器学习系统



推理中产生的KV cache占据大量显存空间



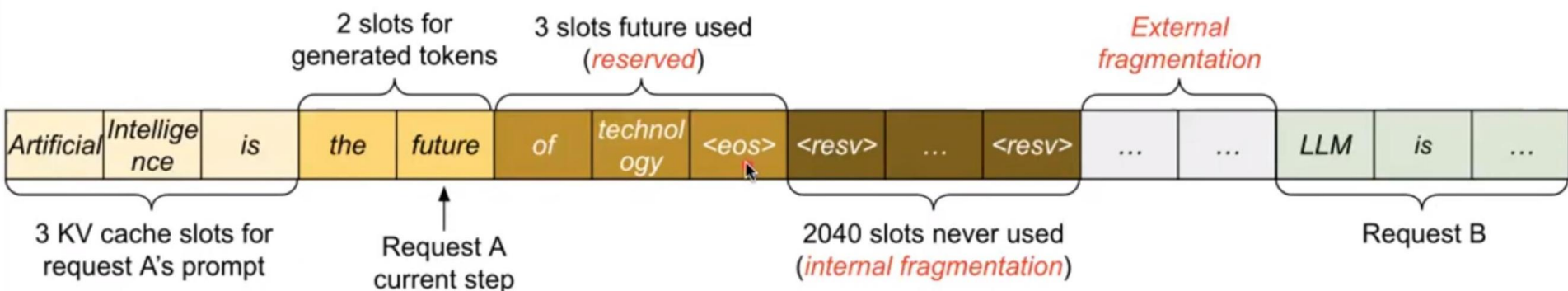
13B LLM on A100-40GB

大模型与机器学习系统



KV cache管理存在空间浪费

- 已预留但尚未使用的空间 (Reserved) : 虽然未来会使用, 但当前暂未使用到。
- 内部碎片 (Internal Fragmentation) : 实际 token 数小于预留长度, 剩余空间浪费。
- 外部碎片 (External Fragmentation) : 不同请求所需内存大小不一, 造成内存块间不连续, 产生碎片。

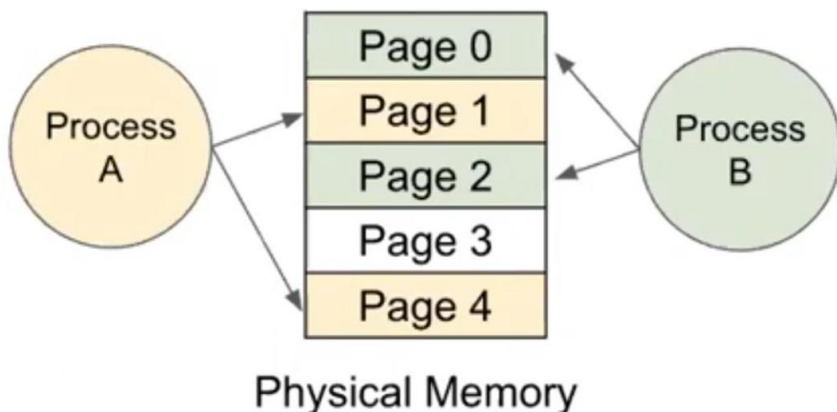


大模型与机器学习系统

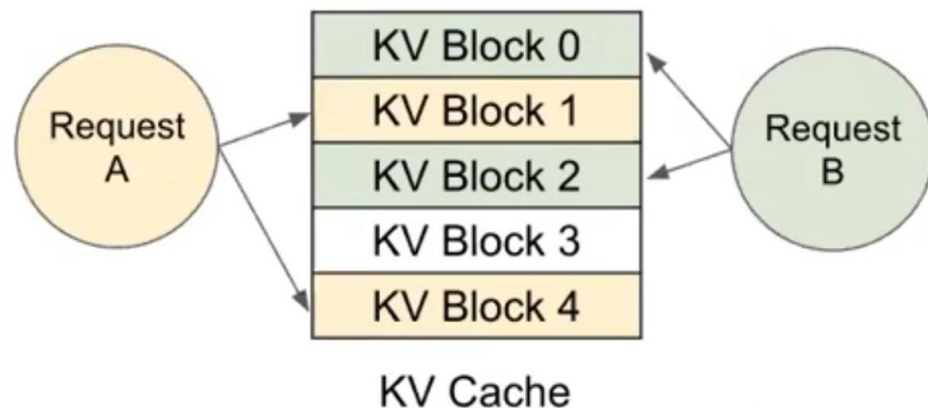


PagedAttention (vLLM) 仿照操作系统的内存页，管理LLM的内存

Memory management in OS



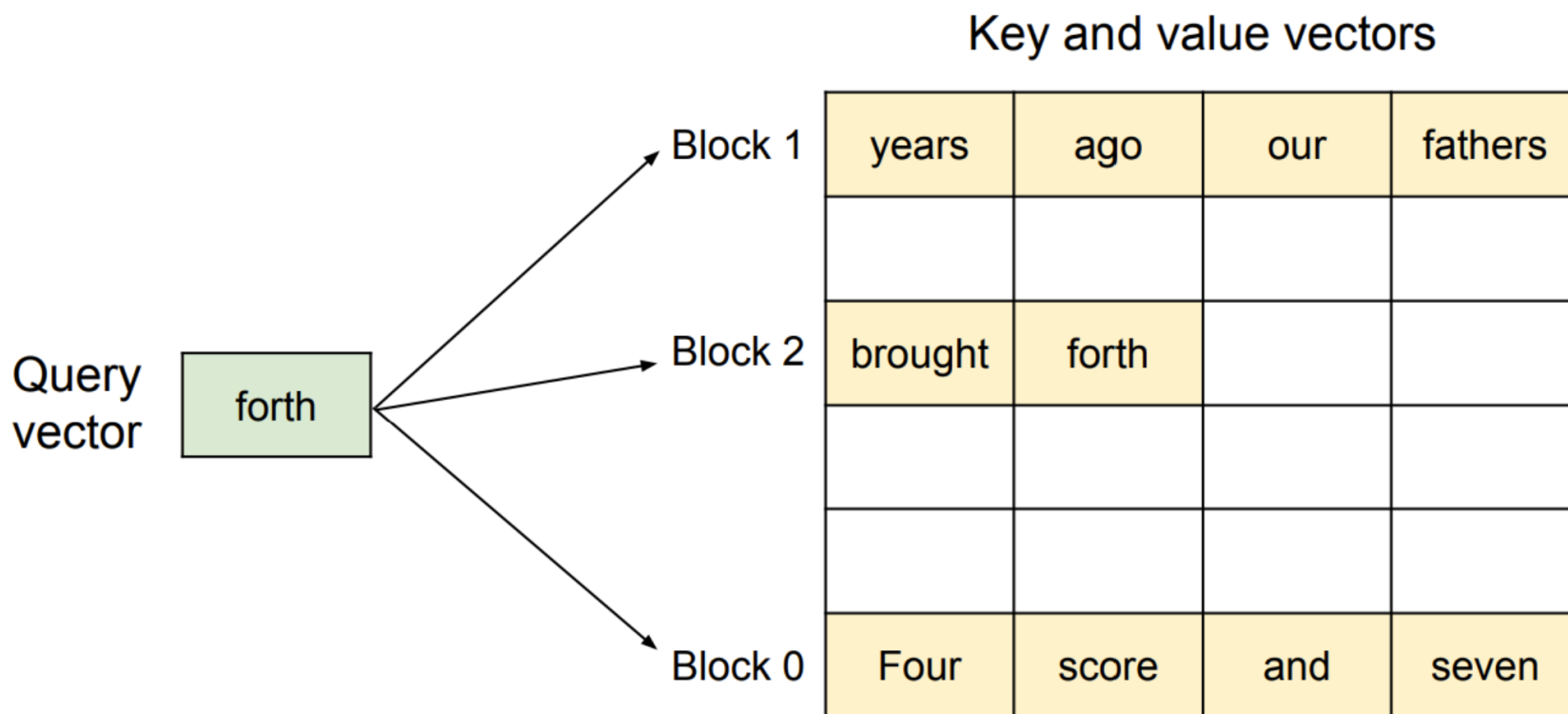
Memory management in vLLM



大模型与机器学习系统



PagedAttention (vLLM) 仿照操作系统的内存页，管理LLM的内存



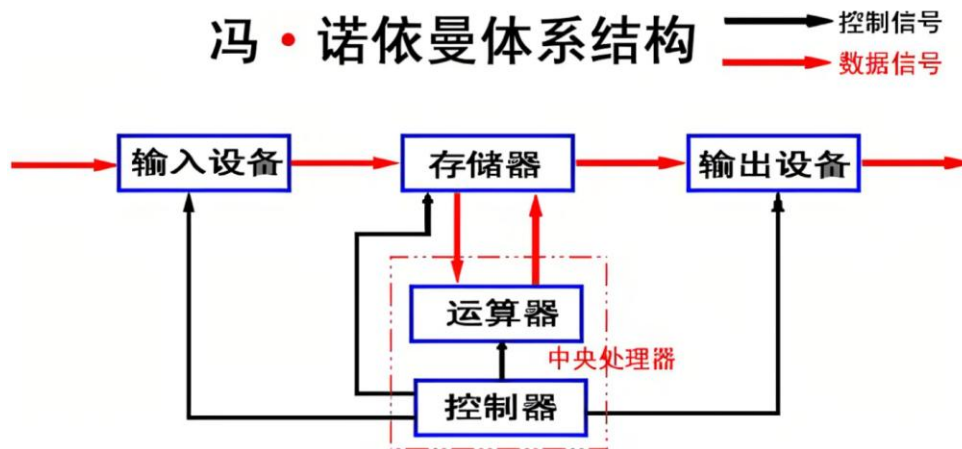
Example sequence: “Four score and seven years ago our | fathers brought forth”



存储和计算的关系



冯·诺依曼体系结构



计算和存储- 冯诺依曼架构最重要的两部分

在体系结构层面-存算分分合合

- 存算一体 - 极致性能
- 存算分离 - 扩展性

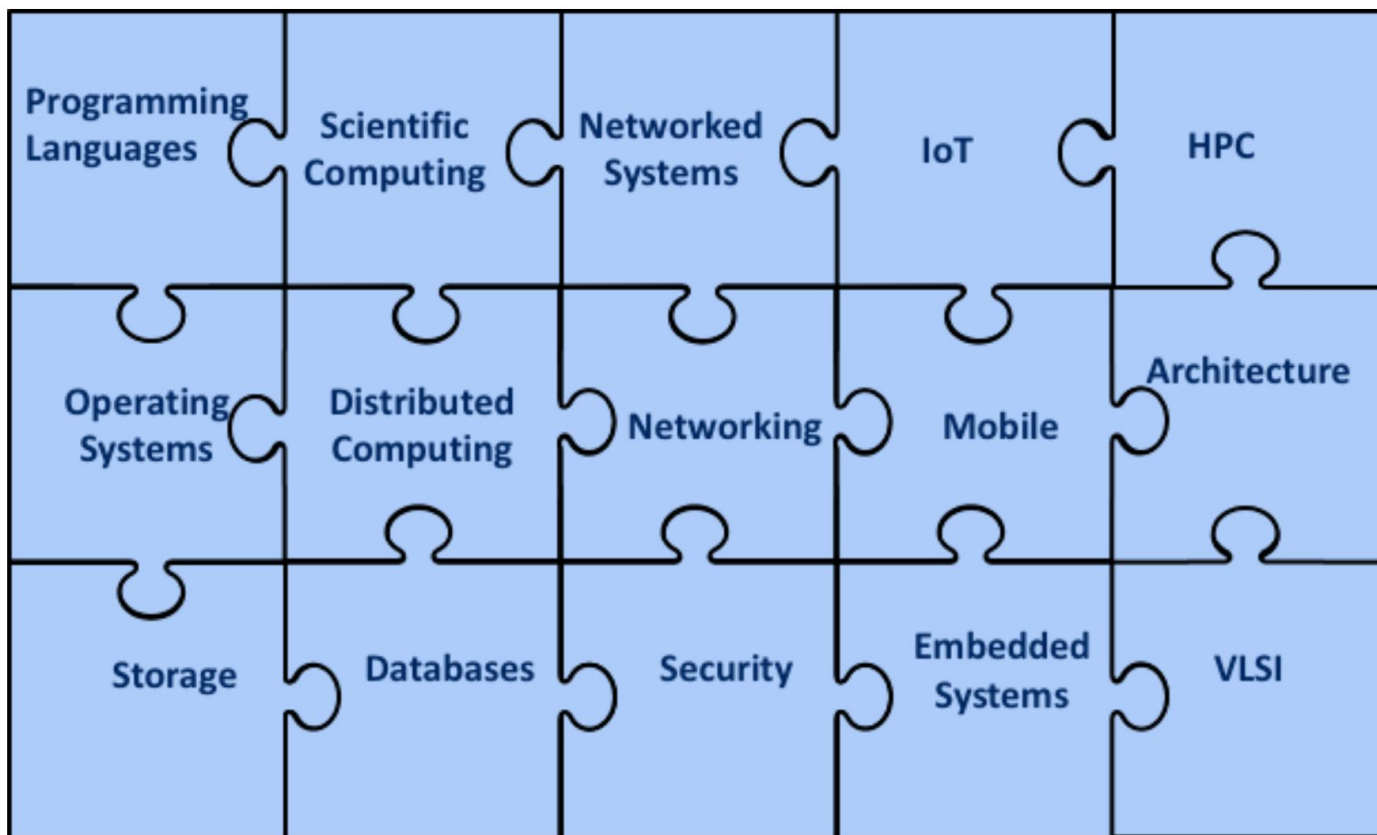
在应用层面-存算相互协同依赖

体系结构层面-分分合合



- 系统结构的分分合合，都离不开存储和计算

Once upon a time, all of systems was one big area; today it looks more like this:



Margo Seltzer

Canada 150
Research Chair

ACM Fellow

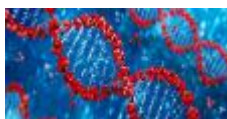
存储和计算不断接近



● 计算任务从主机端到设备端

■ 异构芯片兴起与新型可编程设备出现：融合

➤ 计算密集型 → 数据密集型 → 计算+数据密集型



可编程SSD

- ARM/FPGA

专用数据处理



可编程交换机

- ASIC
- 专用数据包处理



智能网卡

- ARM/FPGA
- 专用数据包处理

存储和计算不断接近

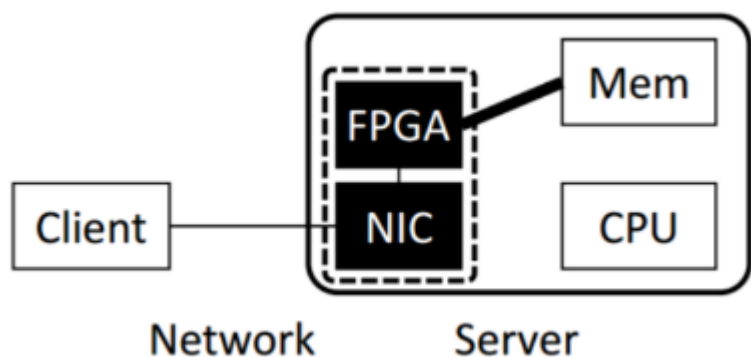
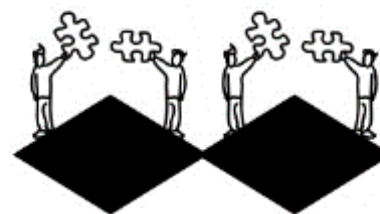


● 计算任务从主机端卸载至设备端

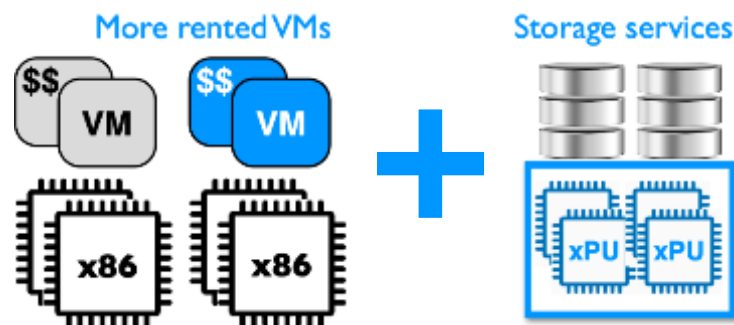
■ 软硬件协同的存储架构

- 让任务在最适合的位置执行
- **I/O栈设计**
- **分布式协议重构**

DOING CO-DESIGN



KV处理逻辑卸载至可编程网卡
[KV-Direct, SOSP'17, MSR]



I/O栈卸载至廉价处理器（ARM）
[LeapIO, ASPLOS'20, Uchicago]

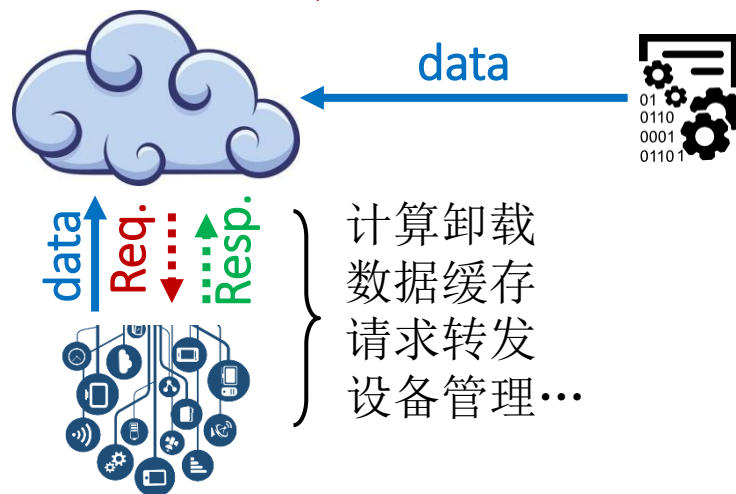
存储和计算不断接近



● 计算任务从云端卸载至边缘端

■ 2020年，全球超过500亿的终端和设备联网

- 物联网技术蓬勃发展（工业互联网、车联网、5G等）
- 超过40%的数据在边缘端产生，总量达16ZB
- **边缘计算/存储：在数据的源头就近处理数据**
- **优势：减少网络开销、降低延迟、提升用户体验（高带宽、低延迟），避免敏感数据跨网传输，更高的数据隐私安全**

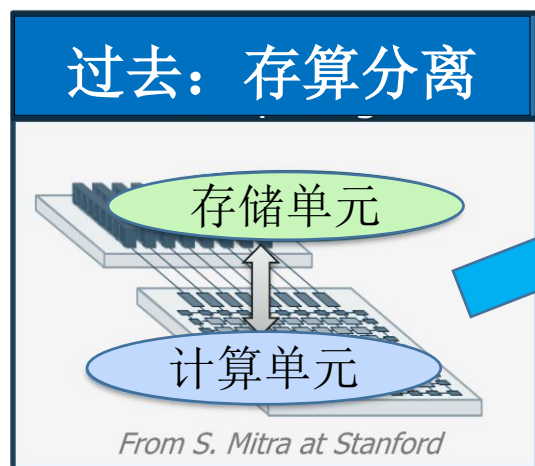


存储和计算不断接近

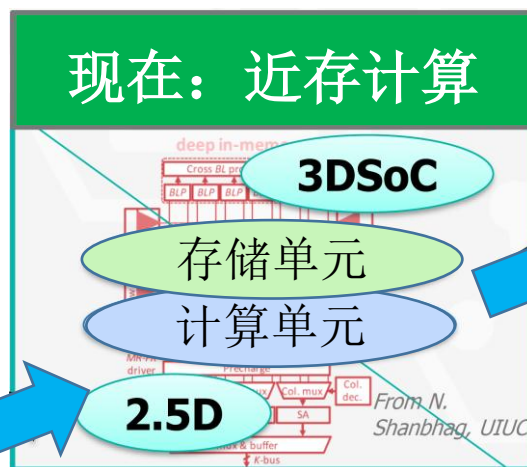


● 设备端的存算融合

- 存储与计算融合的新型体系架构将能有效应对边缘计算与边缘存储对功耗、性能、安全的需求

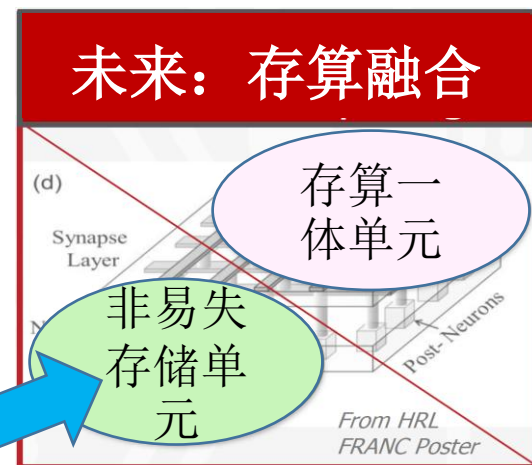


存算分离的传统架构



近存计算的架构

- 三星： SmartSSD
- 阿里巴巴： 可计算存储
- 谷歌： TPU-V2

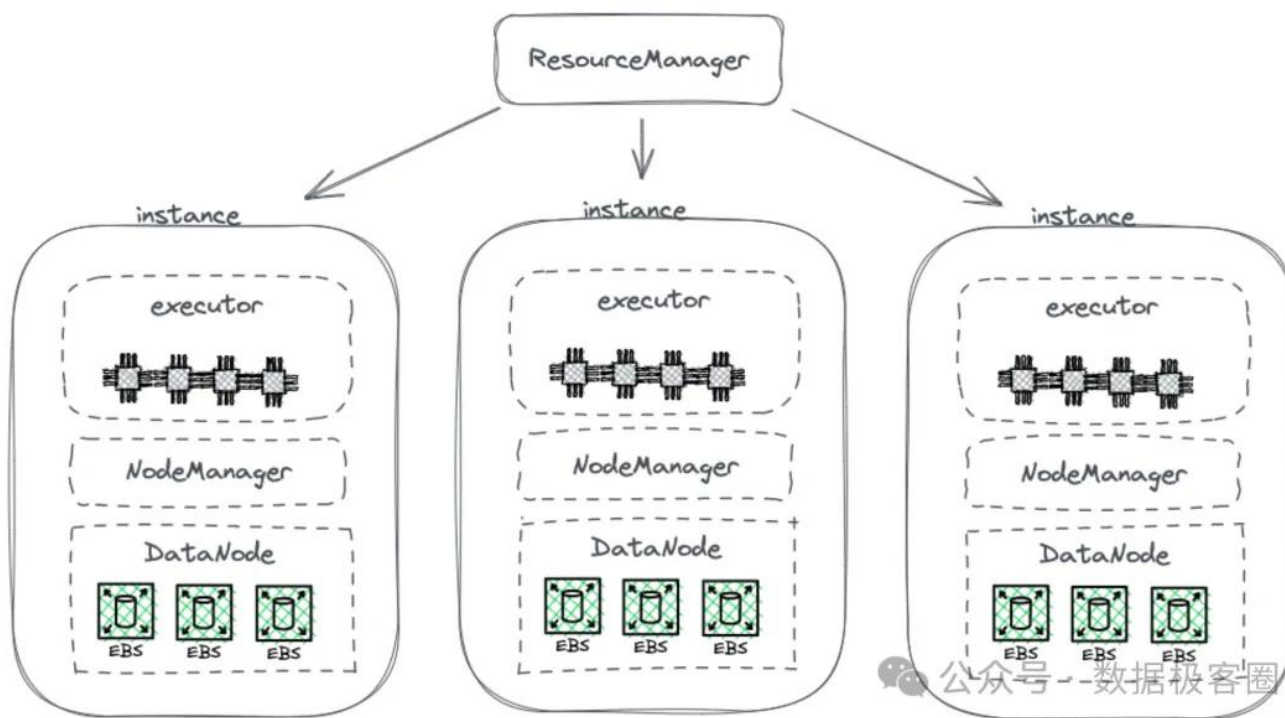


尚在研究的未来架构

计算和存储分离



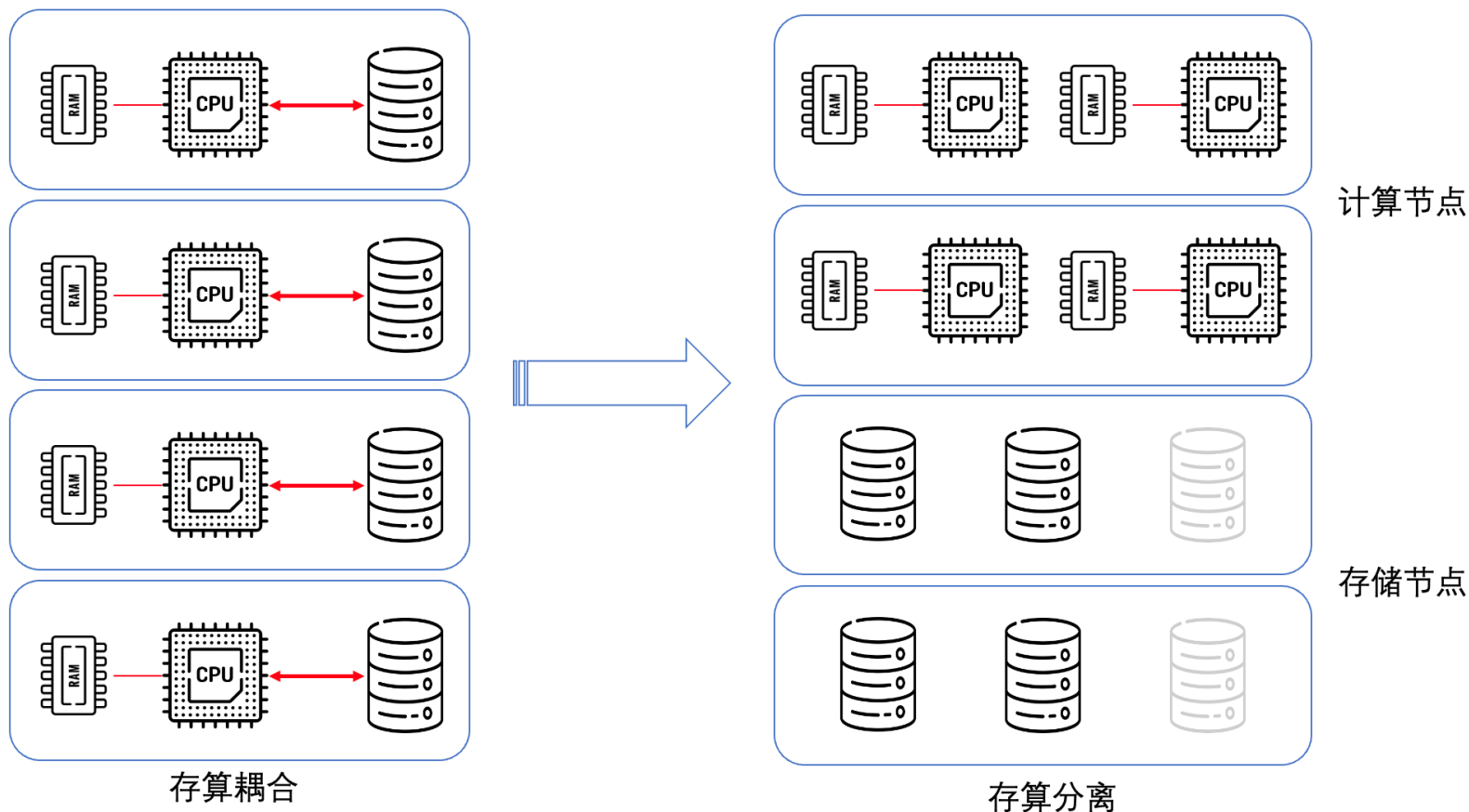
- 分布式系统的早期架构中，存储和计算单元位于同一台机器中，“存算一体”
- 业务场景对计算和存储资源诉求不同，导致资源浪费



计算和存储分离

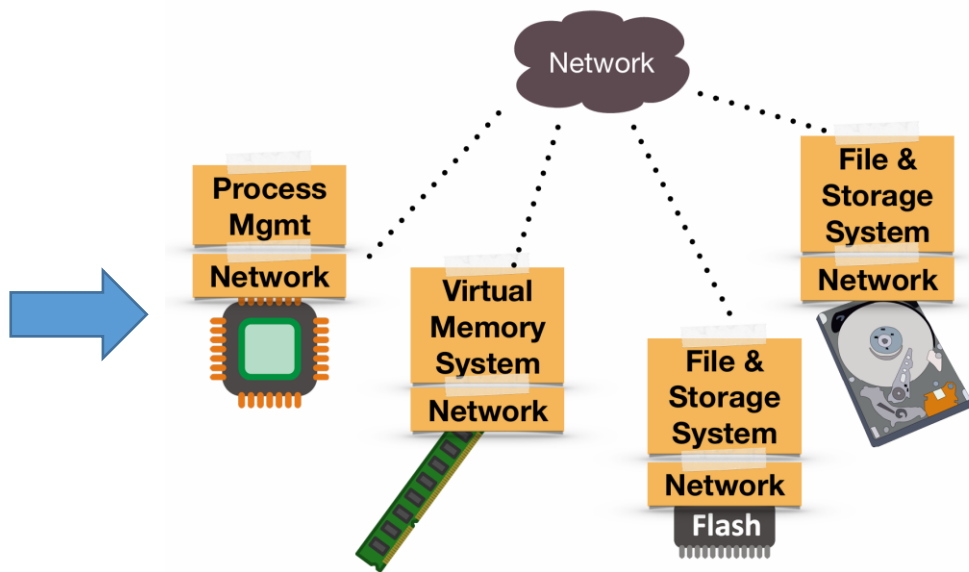
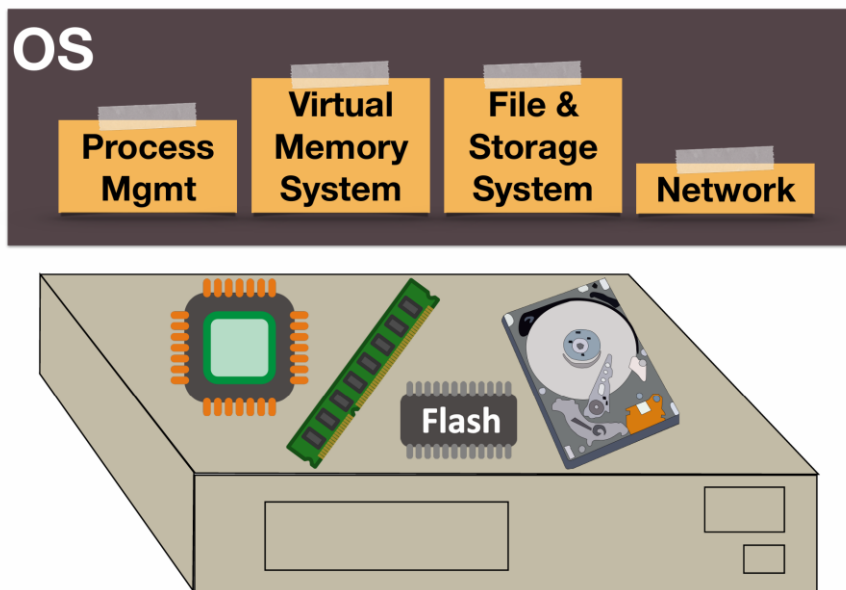


- 存储节点和计算节点分离，各自独立拓展



计算和存储分离

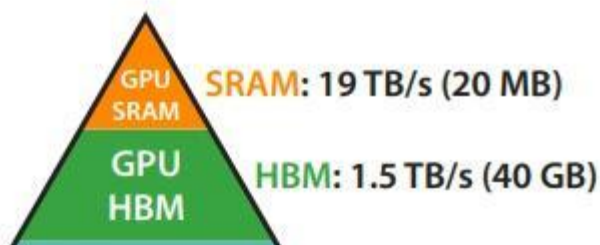
- LegoOS：搭积木的操作系统，计算、内存、存储等资源全部独立拓展



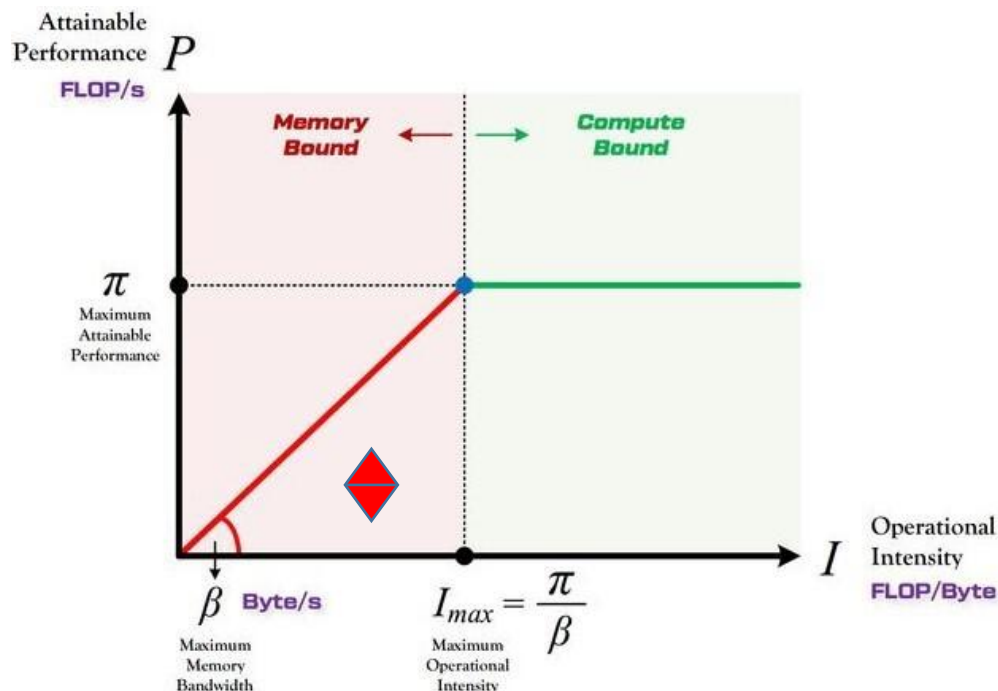
应用层面-相互依赖



Flash attention – 快速的self-attention实现



传统的self-attention需要将数据在global mem(HBM)和local mem(SRAM)间频繁搬运

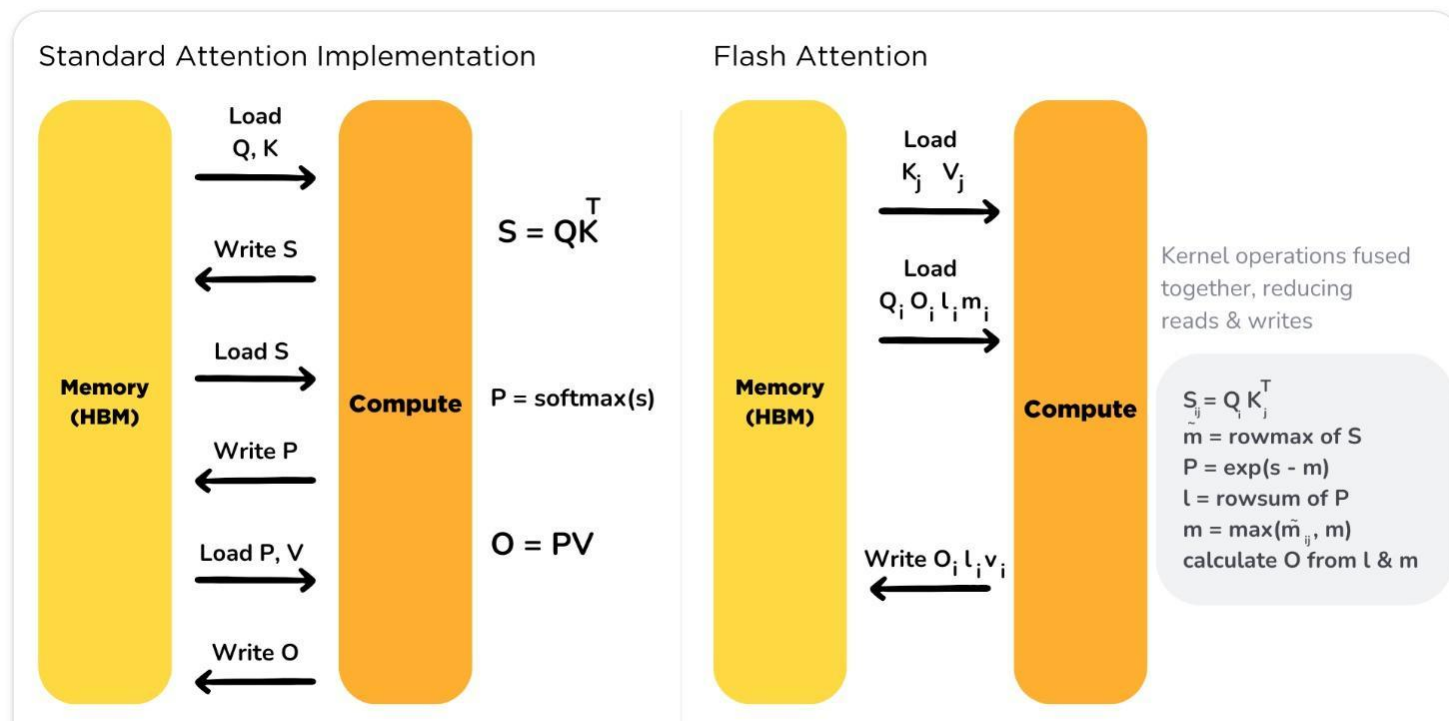


根据roof line model, 属于明显的memory bound操作

应用层面-相互依赖



Flash attention – 快速的self-attention实现



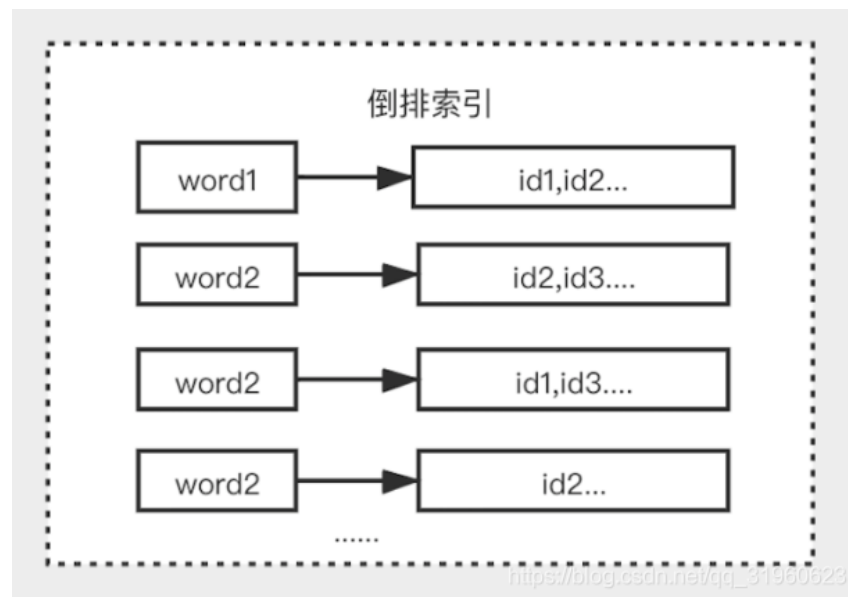
通过分块和重计算，减少对HBM的访问，尽量用SRAM来计算

应用层面-相互依赖



搜索引擎的索引管理- 计算-存储的协调

- 大规模搜索引擎中，倒排索引需要存储在磁盘上
- 不同的压缩方案，比如 pfordelta, bitmap等。
- 高压缩率算法：计算重，I/O轻
- 低压压缩率算法：I/O重，计算轻



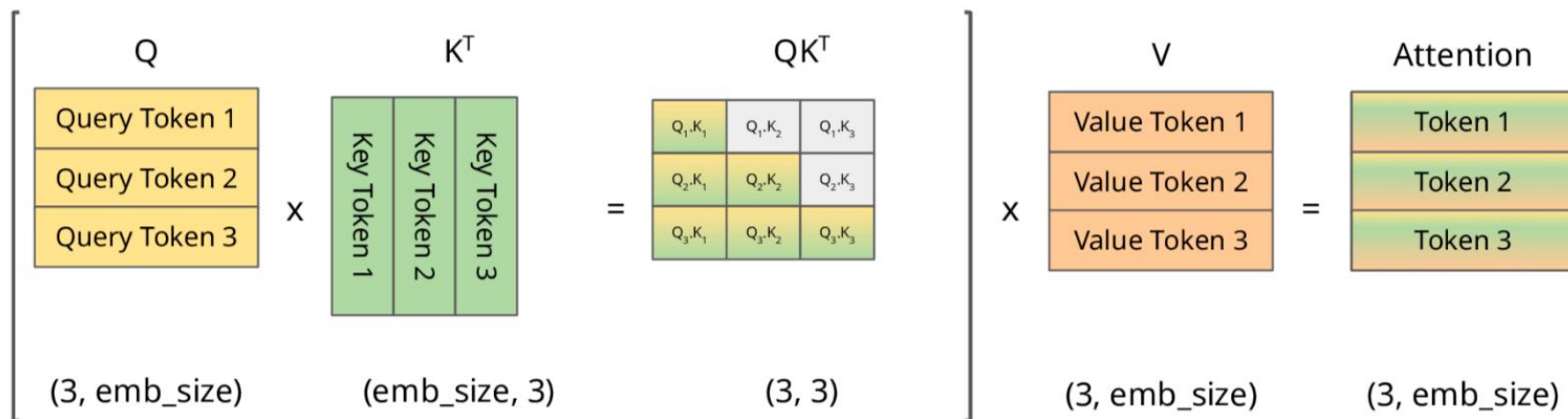
应用层面-相互依赖



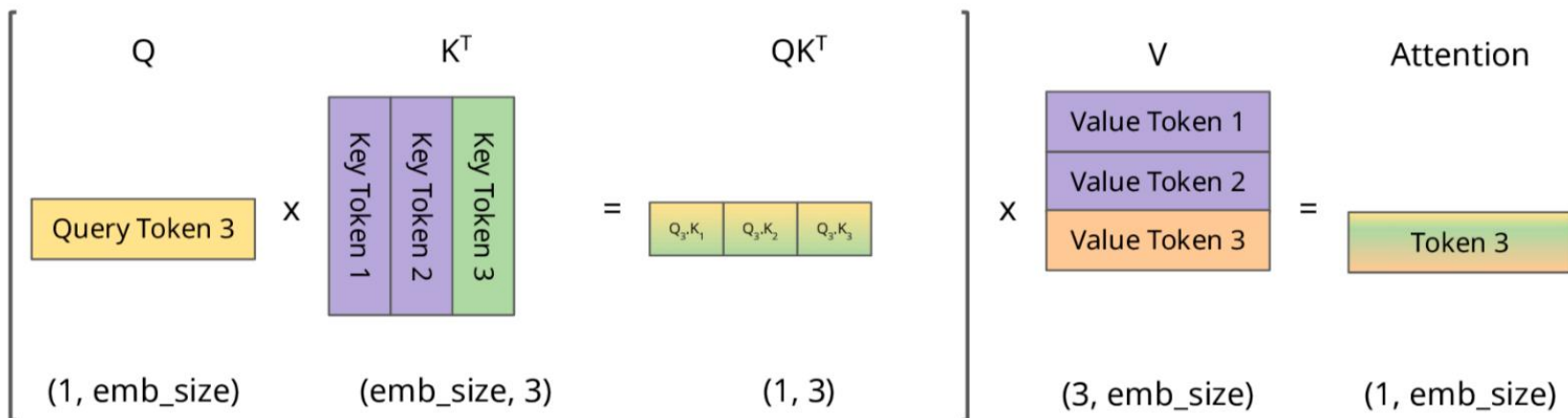
MoonCake – 国产机器学习推理系统

Step 3

Without
cache



With
cache



应用层面-相互依赖

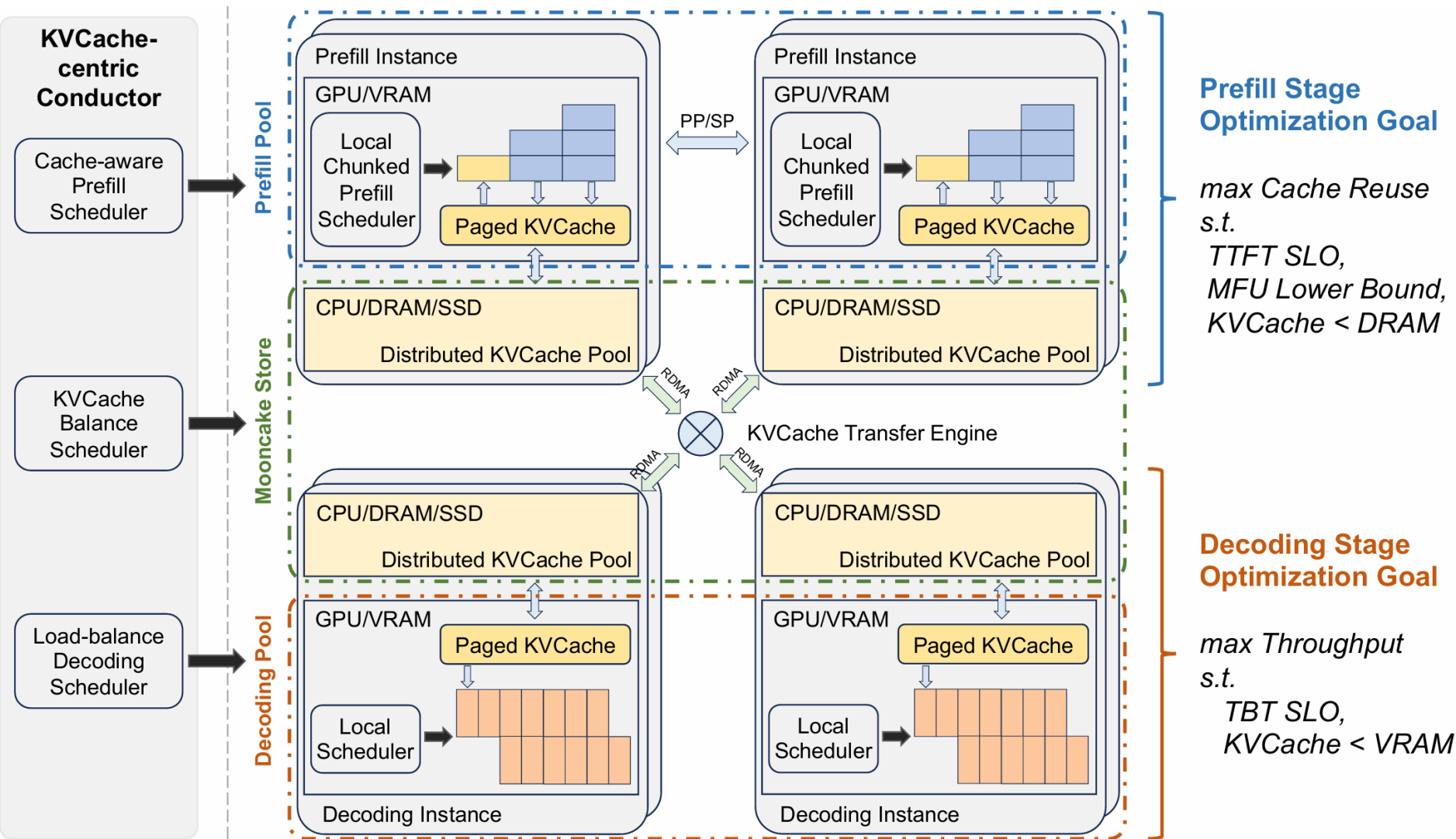


MoonCake – 国产机器学习推理系统

存储为中心的大语言模型推理架构

- 将预填充和解码阶段部署在两个独立集群，并利用高速RDMA网络构建跨节点共享KV cache
- 构建调度器，将预填充阶段产生的KV cache及时调度到解码阶段使用
- 支撑国产Kimi大模型推理服务

应用层面-相互依赖





目录

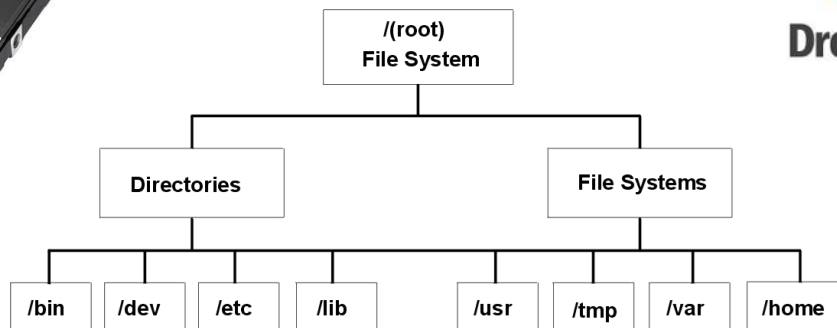
- 课程介绍
- 影响计算机系统设计的因素
 - 硬件变革的影响
 - 软件变革的影响
 - 存储和计算的关系
- 后续课程预告



存储系统



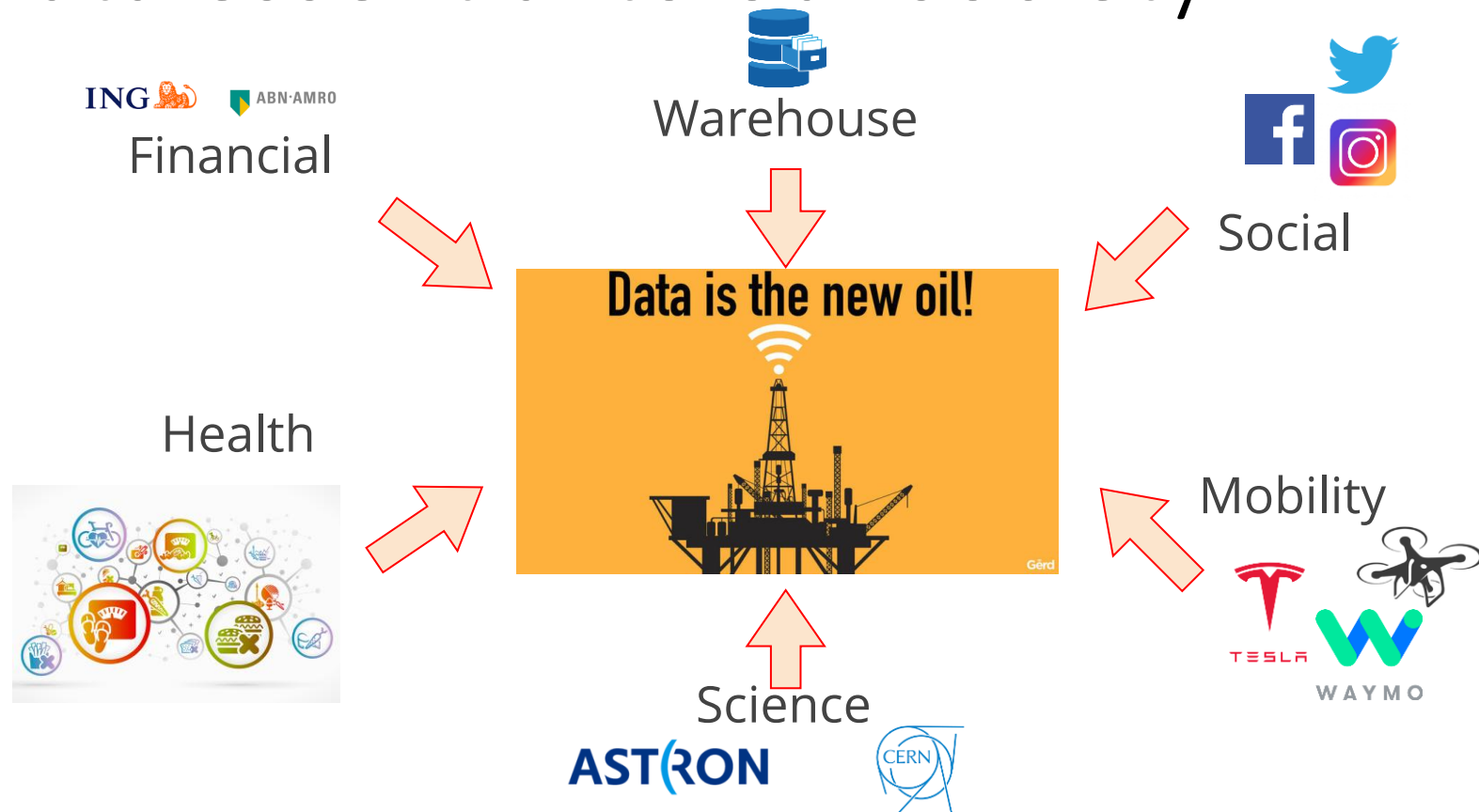
你还能想到哪些？



计算机系统中至关重要的部分
你能想象一台只有计算的计算机吗？

它或许不是计算机中最有趣的部分，但世界上每天都在处理海量的数据

Data essential to our society





ZB并不容易直观感受

假设:

- 1粒米饭是1字节数据



What is big data?, David Wellman, <https://www.slideshare.net/dwellman/what-is-big-data-24401517>

72

假设:

- 1粒米饭是1字节数据
- KB: 1碗米饭



What is big data?, David Wellman, <https://www.slideshare.net/dwellman/what-is-big-data-24401517>

假设:

- 1粒米饭是1字节数据
- KB: 1碗米饭
- MB: 8袋米



What is big data?, David Wellman, <https://www.slideshare.net/dwellman/what-is-big-data-24401517>

74

假设:

- 1粒米饭是1字节数据
- KB: 1碗米饭
- MB: 8袋米
- GB: 3辆卡车



What is big data?, David Wellman, <https://www.slideshare.net/dwellman/what-is-big-data-24401517>

假设:

- 1粒米饭是1字节数据
- KB: 1碗米饭
- MB: 8袋米
- GB: 3辆卡车
- TB: 2艘轮船



What is big data?, David Wellman, <https://www.slideshare.net/dwellman/what-is-big-data-24401517>

假设:

- 1粒米饭是1字节数据
- KB: 1碗米饭
- MB: 8袋米
- GB: 3辆卡车
- TB: 2艘轮船
- PB: 覆盖马斯特里赫特

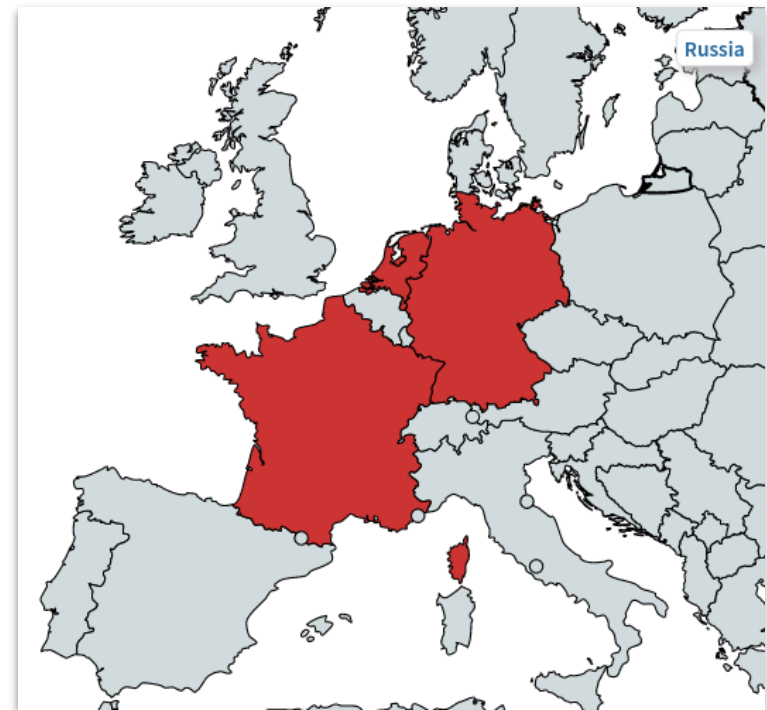


MAASTRICHT



假设:

- 1粒米饭是1字节数据
- KB: 1碗米饭
- MB: 8袋米
- GB: 3辆卡车
- TB: 2艘轮船
- PB: 覆盖马斯特里赫特
- EB: 覆盖德国、法国、荷兰



假设:

- 1粒米饭是1字节数据
- KB: 1碗米饭
- MB: 8袋米
- GB: 3辆卡车
- TB: 2艘轮船
- PB: 覆盖马斯特里赫特
- EB: 覆盖德国、法国、荷兰
- ZB: 填满太平洋



What is big data?, David Wellman, <https://www.slideshare.net/dwellman/what-is-big-data-24401517>



存储系统性能要求严格

我们对实时数据分析的需求日益增长

- 微秒级、毫秒级分析
- 单次查询处理 GB、TB、PB、EB 级数据

归根结底：你从存储设备读写数据的速度有多快，计算速度就不可能超过这个上限

Computer systems, when scaled. become a storage problem. --Zadok's Law (FAST 2026)

未来8周左右，存储系统前沿